



Module Title

Second Year Project

Assessment Type

Year and Semester

2025 Autumn

Assignment Due Date:2026-03-26

Assignment Submission Date:2026-03-26

GitHub Repository URL: <https://github.com/RakshakSigdel/Footstats>

Group Members

London Met ID	Student Name
24045843	Bibek Chaudhary
24046001	Rakshak Sigdel
24046023	Rojesh Thapa
24046049	Samana Upreti

*I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked.
I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.*

Acknowledgement

I would like to express my sincere gratitude to my project supervisor, Mrs. Lenisha Ghimire for her continuous guidance, encouragement and valuable suggestions throughout the development of the project. Her expertise and constructive feedback were instrumental in shaping this project into a comprehensive solution for grassroots sports management in Nepal.

I am also grateful to Itahari International college for providing me the academic environment and learning opportunities that are necessary to complete this project. I also give special appreciation to the local football and futsal communities who shared their experiences and challenges that helps us to understand the real-world problems that this platform aims to solve.

I would like to thanks my team members Rojesh Thapa, Rakshak Sigdel, Samana Upreti, Bibek Chaudhary whose collaboration, technical skills and dedication made the project completed. Each member has contributed their strengths which enhanced the quality of FootStats.

Abstract

This project, titled FootStats – Amateur Football and Futsal management system, was developed to solve the critical problem arises in grassroots sports activities all over Nepal. The existing system relies in traditional and manual process which leads to many problems like miscommunication, lack of performance tracking and financial mismanagement in tournaments operation.

To solve these issues, the proposed system introduces a centralized web application that digitalize the entire amateur sports management. FootStats enables players to track their performance, club admins to manage teams and tournament host to organize events and collect payment and super admins to maintain system.

The project was developed using Scrum methodology that follows an iterative and collaborative approach. The technology stack includes React.js with Vite and Tailwind CSS for frontend, Node.js and Express for backend, PostgreSQL with Prisma ORM for database management. Socket.IO for real-time chat functionality and Esewa and Khalti API integration for secure payments. The system also implements JWT based authentication for security and Cloudinary for file storage.

The final outcomes include a fully functional web application with role-based access control, CRUD operations for player, clubs, tournaments and matches, real-time group chat for club members, secure payment, match scheduling, statistics tracking and location-based discovery features. The system is expected to provide practical, user-friendly and scalable solution that improves accuracy, accessibility, transparency and overall efficiency in managing community level football and futsal activities in Nepal.

Table of Contents

1	Introduction	1
1.1	Introduction to Project	2
1.2	Introduction to client	3
1.3	Problem Scenario	4
2	Literature Review	5
2.1	Timeline of report/Project Gantt chart	6
2.2	Project As a Solution	9
2.3	Aims and Objectives	9
2.4	Outcomes and Deliveries	10
2.5	Similar Projects	11
2.5.1	Project 1: SportEasy	11
2.5.2	Project 2: LeagueRepublic	13
2.5.3	Project 3: 360Player	15
2.5.4	Similar System Comparison	17
2.5.5	Critical Evaluation	18
2.6	Review Summary	19
2.7	Project risks, threats and contingency plan	20
2.7.1	Risk and Threats	20
2.7.2	Contingency Plan	21
2.7.3	Risk Management Table	22
2.8	Resource Requirements	23
2.8.1	Hardware requirements	24
2.8.2	Software requirements	24
3	Methodology	25
3.1	Considered Methodology	26
3.2	Chosen Methodology	26

4	System Design	27
4.1	System Architecture	28
4.2	Entity Relationship Diagram (ERD)	30
4.3	Use Case Diagram	32
4.4	Flowchart	35
5	Software Requirements Specification (SRS)	38
5.1	Functional Requirements	39
5.2	Non- functional Requirements	41
6	Testing	42
6.1	Authentication (Register)	42
6.2	Authentication(Login)	43
6.3	Club Creation	44
6.4	Tournament Creation	46
6.5	Challenge Club	48
6.6	Accept Challenge & Schedule Match	49
6.7	Club Chat System	51
6.8	Match Event Recording	52
6.9	Notification System	54
7	Group Contribution Table	55
8	Conclusion	57
8.1	Legal Issues	58
8.2	Social Issues	59
8.3	Ethical Issues	59
8.4	Future Work	59
9	References	60
10	Appendix	61
10.1	Code Snippets	61

10.2	User Survey Questions	67
10.3	API Endpoints Specification	68
10.4	User Stories	69
11	Developed Project (Screenshots / System Demo / Wireframe)	70

Table of Figures

Figure 1 Gantt Chart.....	7
Figure 2 System Architecture	28
Figure 3 ERD	30
Figure 4 Use case diagram	32
Figure 5 Use case diagram	33
Figure 6 Flowchart.....	36
Figure 7 Register User Test	42
Figure 8 Login Test	43
Figure 9 Creating Club	44
Figure 10 Club Created	45
Figure 11 Creating Tournament	46
Figure 12 Tournament Created	47
Figure 13 Challenging the club during schedule creation.....	48
Figure 14 Match request is received by another club admin	49
Figure 15 Matches are scheduled	50
Figure 16 Chat System test.....	51
Figure 17 Add match event	52
Figure 18 Match event added successfully	53
Figure 19 Notification for new schedule	54
Figure 20 Notification through email.....	54
Figure 21 Backend Entry Point and Route Wiring	61
Figure 22 Prisma Schema.....	62
Figure 23 Auth Controller	63
Figure 24 Schedule Controller.....	64
Figure 25 Frontend App Routing Skeleton	65

Figure 26 Frontend API Client with Auth Interceptors	66
Figure 27 Login page	70
Figure 28 Register Page	71
Figure 29 Create Club or Join	71
Figure 30 Club Details.....	72
Figure 31 Chat Section.....	72
Figure 32 Host or Join Tournament.....	73
Figure 33 Tournament Management.....	73
Figure 34 Create Schedule	74
Figure 35 Manage line-ups.....	74
Figure 36 Update Match Event.....	75
Figure 37 Player Statistics Chart.....	75
Figure 38 Profile View	76
Figure 39 Update Profile	76
Figure 40 Payment.....	77

Table of Table

Table 1 Similar System Comparison	17
Table 2 Risk Management Table	22
Table 3 Functional Requirement	40
Table 4 Non-Functional Requirements.....	41
Table 5 Test case of registration	42
Table 6 Test case of Login	43
Table 7 Test case of club creation	44
Table 8 Test case of Tournament Creation	46
Table 9 Test case of Challenge Club	48
Table 10 Test case of Accept challenge and schedule match.....	49
Table 11 Test case of Club Chat System	51
Table 12 Test case of Match Event Recording.....	52
Table 13 Test case of Notification System	54
Table 14 Contribution Table	56
Table 15 API Endpoints.....	68

1 Introduction

This chapter introduces the FootStats, a web application and the context in which it was developed. It explains the background of grassroots sports management challenges in Nepal, identifies the key stake holders affected by current inefficiencies and outlines the need for better digital solution. This chapter provides a detailed overview of the project scope, client identification and the problem scenario that motivated FootStats team for development. The following pillars are important to understand how the concept of FootStats can be used to satisfy the practical requirements of amateur football and futsal communities in Nepal.

1.1 Introduction to Project

The FootStats is a comprehensive web-based sports management system that is developed to make a revolution in the amateur football and futsal activities that are organize and managed in Nepal. The project belongs to the domain of sports technology and community management, where timely access to accurate information, efficient coordination and transparent operations are critically important for the success of grassroots sports initiative.

The systems provide multiple user group including individual players who participate in local matches, club administrators who are responsible for team management, organizing tournament and super administrators who maintain system integrity and others operations. Each user role has been designed to address specific area of concern in the existing manual process of managing sports activities.

The growing popularity of football and futsal in Nepal, particularly among students' clubs and local community players, has created an urgent need for digital transformation in this sector. Many people who are actively participate in local matches, intercollege tournaments and other community leagues, making grassroots sports an integral part of daily life. However, the infrastructure supporting these activities has not kept with this growth, creating huge number of operational challenges.

The proposed FootStats aims to replace traditional informal communication method with a reliable, user-friendly and organized platform that connects all the stakeholders in the amateur sports ecosystem. The project offers a feature such as player statistics tracking, club management tools, tournament registration with a secure payment processing, match scheduling and a real time communication. FootStats creates a complete digital ecosystem for grassroots sports management. This platform is designed with a scalability mind which allows it ti grow from serving a local community to potentially supporting regional and national amateur sports network.

1.2 Introduction to client

The primary clients and beneficiaries for the FootStats are the grassroots football and futsal communities across Nepal that includes club administrators, tournament organizers and sports coordinators. FootStats is not like the traditional client-developer type of relationship where a single organization outsources software development but serves the needs of a larger sports ecosystem that does not have sufficient digital infrastructure at the moment.

- i. **Amateur Players:** These users are the largest segment of the client base that includes students, young professionals and community members who participate in local football and futsal matches. They are currently facing problems in monitoring their performance statistics, finding local clubs and tournaments and being aware of the match schedules. Players require a centralized system on which they can see their goals, appearances, achievements and compare their performance with others using leader boards.
- ii. **Club Administrators:** Club admins will perform the duties of the management of teams, the organization of practises, matches, and the records of the members. They are now grappling with manual management of the members, inefficiencies in their communication as well as the absence of applications to manage the performance of teams. Such customers need club creation, member approval systems, team statistics tracking, and internal communication systems.
- iii. **Tournament Organizers:** Tournament organizer hosts competitive tournaments ranging from small inter club matches to large tournaments. They face many challenges while registration, collecting entry fees and generating fixtures, scheduling matches and communicating with participating teams. These clients need club creation features, member approval systems, team statistics tracking features, and internal communication tools.

The FootStats was designed based on needs of clients, skills and scalability. We also gathered insights with players, managers and organizers in Itahari that shaped system's design and features.

1.3 Problem Scenario

In today's, the management of amateur football and futsal activities in Nepal is performed through phone calls, WhatsApp groups and verbal communications which creates several operational problems. The lack of a specific digital platform tailor-made to manage sports at the grassroots level has led to systematic issues that guide how players, clubs, and organizers of tournaments operate every day. Some of the problems are mentioned below:

- i. **Communication issues:** Match schedules, venue changes and other announcements are shared through fragmented social media groups, leading to missed information and miscommunication between the players and clubs.
- ii. **No Performance Tracking:** Players cannot track their goals, assists or others systematically. There is no any record of individual or team statistics that prevents players from showcasing their abilities and measuring improvement over time.
- iii. **Scheduling Conflicts:** Manual coordination results in double booking, incorrect venue information, forgotten matches and incomplete teams due to lack of automated reminders.
- iv. **Tournament Managing problems:** The organizers face the problem of manual registration process, collecting entry fees through informal methods and generating fixtures.
- v. **Limited Club Discovery:** Players have no platform to find clubs nearby or submit join request to the club. This process relies entirely on personal networks that limits opportunities for newcomers.
- vi. **Payment Transparency Issues:** Tournament fees are collected through cash or individual digital transfers with no standardized receipts, creating accountability problems and reducing trust.
- vii. **Lost Records:** The important match results, player achievements and historical data are lost overtime as they are stored in personal devices or chat histories rather than a permanent database.

The above issues, user experiences delays, reduced productivity, poor decision making and missed opportunities. This situation shows the need for a centralized technology-based solution that can digitalize grassroots sports management in Nepal.

2 Literature Review

The literature review provides the theoretical and practical background that are necessary for the development of FootStats. It examines relevant concepts, existing sports management solution and related studies in the field of grassroots sports technology and community platform development. By reviewing previously developed systems and current industry practices, this chapters shows effective approaches, common limitations and opportunities that guide the design of proposed solution. This review demonstrates that FootStats is built on a informed research rather than assumptions, focusing that the systems will solve real needs with proven methodologies.

2.1 Timeline of report/Project Gantt chart

The project timeline was planned to ensure systematic completion of all major activities within the allocated academic period. The work was divided into seven sprints following scrum methodology. The phases include problem identification and requirement analysis, literature review, system design, database schema development and backend implementation with API development, frontend development and UI/UX implementation, integration testing and bug fixes and final documentation with deployment preparation.

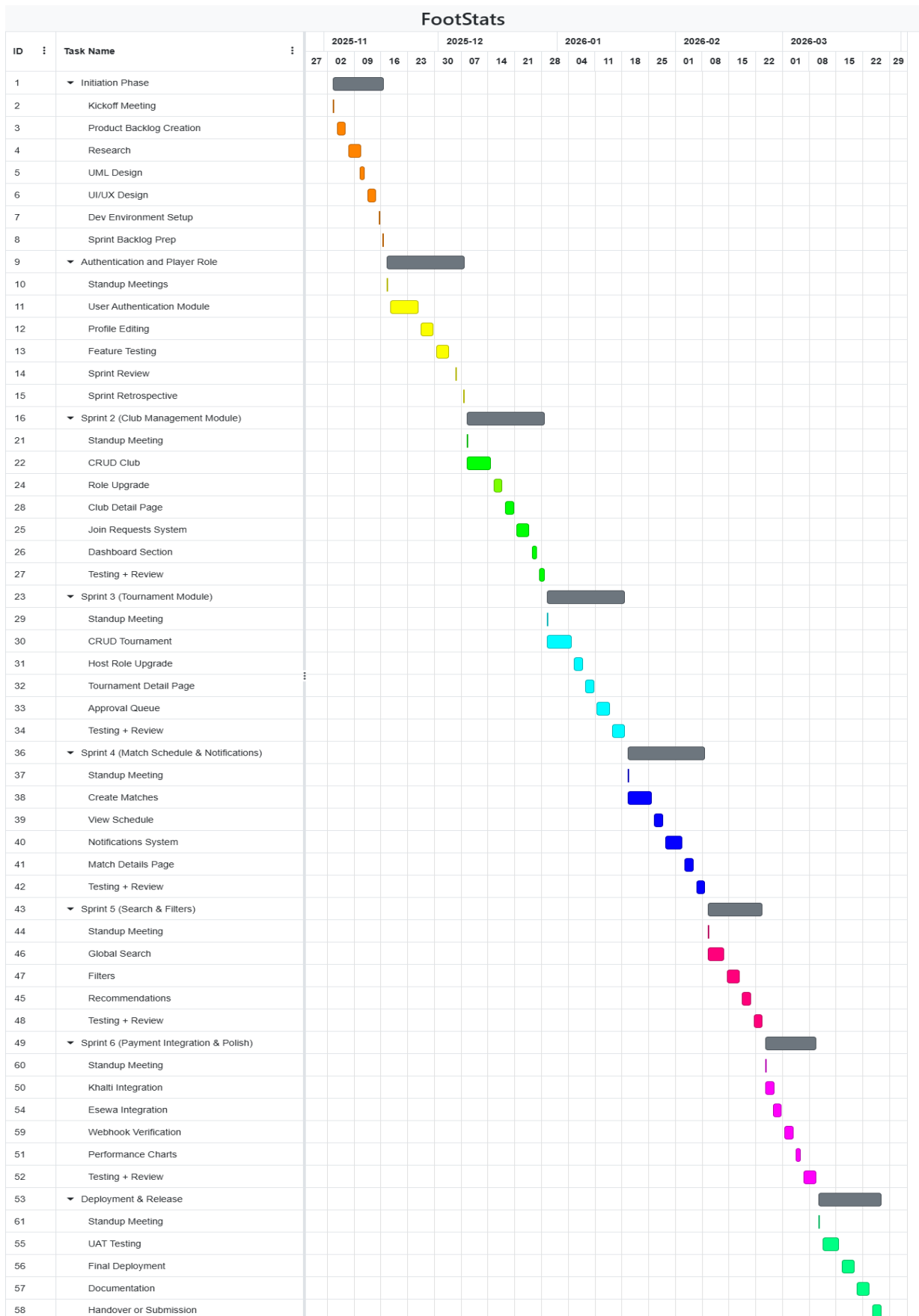


Figure 1 Gantt Chart

- i. Initiation phase provides project foundation through idea finalization, feasibility study, research on stack, system requirements gathering, sprint planning, development environment setup and sprint review.
- ii. Sprint 1 focuses on database architecture with Prisma schema modelling, GitHub repository setup, user registration and login implementation and initial feature rollout.
- iii. Sprint 2 implements core crud operation for player, clubs, tournaments including testing of each module.
- iv. Sprint 3 develops Realtime club chat functionality using socket.io, club members joining, tournament registration logic and sprint review.
- v. Sprint 4 creates match scheduling functionality, implements match status management, view schedules features, player statistics and sprint review.
- vi. Sprint 5 builds leader boards, UI/UX refinement, push notification for reminders, feature testing and sprint review.
- vii. Sprint 6 integrates Esewa and Khalti payment gateways implements payment verification, final code clean-up and sprint testing.
- viii. Sprint 7 handles deployment to production, bugfixes, feature testing on live environment and final submission.

2.2 Project As a Solution

The proposed project, FootStats, serves as a practical solution to the management faced by amateur football and futsal communities in Nepal by transitioning from informal, manual process to centralized digital platform. It replaces ad-hoc communication methods such as phone calls and groups chat with a structured web application that ensures transparency and better coordination. The system provides core functionalities including real-time tracking of player and club statistics, match scheduling with notifications and location-based discovery of nearby sports activities. Additionally, by integrating secure payment gateways like Esewa and Khalti, the project also solves issues related to fee collection and financial mismanagement. This digital approach directly addresses the problems of miscommunication, loss data and scheduling conflicts making grassroots sports management more efficient and accessible for all stakeholders (Thapa, 2025).

2.3 Aims and Objectives

The main of this project is to design and develop reliable, user-friendly and scalable web-based platform that enables comprehensive management of amateur football and futsal activities in Nepal, addressing critical gaps in communication, organization and record keeping that currently hinder the growth and efficiency of grassroots sports communities. To achieve the aim, the project should focus on the following specific objectives.

- i. In order to develop a role-based access control system that distinguishes the user privileges of Players, Club Admins, Tournament Hosts, and Super Admins.
- ii. To create a statistic tracking module capable of properly tracking and reporting on player statistics like goals, assists, wins, and match history.
- iii. To install a match scheduling and fixtures generation computerized system that can send a notification and reminders to registered users.
- iv. To ensure secure payment gateways (Esewa, Khalti) are built in to enable fair and credible fee collection of tournaments registration.
- v. To develop a location-based discovery functionality where users can search and filter nearby clubs, teams and tournaments to their geographical location.

2.4 Outcomes and Deliveries

The anticipated deliverables of this project will be a complete sports management system (web based), properly designed PostgreSQL database and an easy-to-use interface screens of various user roles. The most important deliverables consist of the entire source code published on GitHub, the system design (ERD, Use Case) diagrams, and the documentation of the Software Requirements Specification (SRS) document and project report. Moreover, the project will provide integrated tournament management, statistics tracking, and secure payment processing modules, which can be viewed as a viable digital environment solution to the local sport community.

2.5 Similar Projects

There are a few available systems connected with sports management, football clubs' management, and tournament management, which were examined to know new trends and new features. In these systems, valuable information on the user interaction, match scheduling, tracking of statistics and payment is made available. They however indicate shortcomings like failure to support localized payment gateways, community level discovery capabilities and inaccessibility to Nepalese grassroots users. Learnings of such projects can weigh FootStats on a firmer ground and enable it to design the best choices to meet Nepali amateur football and futsal community.

2.5.1 Project 1: SportEasy

SportEasy is an online and mobile solution used by many amateur sport groups and clubs to aid in their day-to-day management. It was first introduced in Europe, and now is used by coaches, administrators, players, and parents of soccer, futsal and basketball. It comes as a web-based application and as both iOS and Android native applications (SportEasy, 2024).

Purpose and Main Functions

SportEasy also eliminates the management of sports clubs that are amateur in nature, by centralizing communications related to the organization, competition, statistics of these players and how to collect the recommended sum of money through a platform. It helps coaches in scheduling the matches and training sessions, keeping the record about the attendance and helping to record goals, assists and cards.

Strengths

SportEasy has strong team involvement and daily communication component. This is a time saver in terms of time spent on administration because of its automatic reminders and invitation system. The non-technical coaches and volunteers can access the simplicity and user-friendly interface. It is also the sponsor of several sporting activities and boasts a huge following base.

Limitations

SportEasy is not designed to manage at a tournament level, it does not even have fixtures generation, group stages or knockouts brackets. It does not support such local payment gateways as ESewa or Khalti, which is inappropriate to use in Nepal. There is no geographical discovery of clubs or tournaments around. Its platform is also not a cross-club leader board, which is incompatible with the grassroots football and futsal in Nepal and its platform is based on the model of Western sports clubs.

Key Features

- Communal team calendar and schedule of matches and training.
- Tracks of player statistics such as goals, assists, cards and played minutes.
- Chats within the app instead of informal group chats.
- Membership and event payments Online.
- Invitations with automatic notifications of attendance.
- Formation support and MVP voting Line-up creation.
- Multi team club management module in a single association.
- Youth team management parent access portal.

2.5.2 Project 2: LeagueRepublic

LeagueRepublic is a web-based sports league management system which has been operational since 2002. It is common throughout the UK, and the English Football Association (FA) has placed its confidence in it to carry out the administration of grassroots leagues. It is the play host of thousands of organizations such as amateur and semi-professional football leagues and school tournaments. The basic features are available without charge with premium plans being offered in accordance with the number of team members (LeagueRepublic, 2024).

Purpose and Main Functions

LeagueRepublic revolves around the process of league administration and tournaments as viewed as a league secretary or organizer. It also computerizes the process of scheduling fixtures, automatically maintaining standings and results as well as giving publicly free league sites in which fans and players can track their respective teams.

Strengths

LeagueRepublic is extremely effective in league generation and league standings. It has sophisticated schedule to deal with tricky conflicts such as the double-booking of a venue. Most small amateur leagues will be able to manage with the free version. It has good credibility in the grassroots football communities, because of its long existence and FA adoption.

Limitations

LeagueRepublic is not created with the idea that an individual player or club managers would be supported; instead, it is meant to support league administrators only. It does not have a player dashboard, club managers do not have role-based access, and there is no in-app messaging. The platform does not provide any kind of location-based discovering of clubs or tournaments. The payment processing is purely supplied with Stripe which is not an option in Nepal. Mobile experience is not maximized, and interface has a learning curve that is quite challenging to young or non-technical users.

Key Features

- Automated scheduling of fixed assets using higher-order scheduling tools.
- Round-robin support, knockout support, and group stage support.
- The website of public league generated automatically per league.
- Registration of teams and players online through Stripe-based payment.
- Team administrators should be able to enter results in the form of approval workflows.
- Goals, assists, cards, and appearances of a player.
- Kick-off tables with adjustable tiebreaker policies.
- Social media and email of match updates.

2.5.3 Project 3: 360Player

360Player is a complete to-sports club management solution with a focus on football, futsal, and basketball among other games worldwide. In contrast to SportEasy, a team management tool, or LeagueRepublic, a league management tool, 360Player is positioned as a team management tool (for coaches and administrators alike). It provides a web-based application and iOS and Android mobile applications. (360Player, 2024)**Purpose and Main Functions**

360Player can be configured to support the entire lifecycle of a sports club, including member registration and collection of payments all the way to tracking individual player development. It helps facilitate various functions such as players, coaches, parents, and club administrators and provides statistical dashboards, video analysis and a personal development plan.

Strengths

The depth analysis of player performance and the individual development tools used in 360 players are the greatest strength of the product. Its powerful multi-role will make players, coaches, parents and admins feel their own experience. Auto payment and invoicing features save club administrators with financial management overhead.

Limitations

360Player is not made to suit individual players that are looking to find a new club or local tournaments. It does not have a location-based search or a community discovery feature. Its price is too high to be bought by amateur clubs in Nepal. It is not connected to local payment systems, such as ESewa or Khalti. Neither does it provide a complete tournament organizer's experience with fixture generation and public tournament discovery. Its complex capabilities are additional and can confuse users on an informal community level.

Key Features

- Granting access to players, coaches, parents and administrators based on their roles.
- Statistics on performance of the players such as goals, assists, wins, and cards.
- Visual player or season performance trend analysis and player or season visual dashboards.
- Auto scheduling and training in match and training.
- Club fee and registration invoices and payments are automated.
- Personal development strategy and coach feedback application.
- In-app social and communication functions.
- Multi-sport sponsorship in football, futsal and basketball.

2.5.4 Similar System Comparison

The three reviewed systems were compared against the proposed FootStats platform based on some key criteria. To briefly capture such a comparison, a table below is given:

Criteria	SportEasy	LeagueRepublic	360Player	FootStats
Main Purpose	Team scheduling & communication	League fixture management	Club member & performance management	Community sports management for football & futsal in Nepal
Key Features	Calendar, stats, messaging, payments, attendance	Fixture generator, standings, result entry, public league site	Role dashboards, player analytics, invoicing, development plans	Tournament creation, player stats, leader board, local payments, discovery
Payment Support	Stripe only	Stripe only (1% fee)	Stripe-based invoicing	ESewa & Khalti
Location Discovery	Not available	Not available	Not available	Location-based club & tournament search
Key Limitation	No tournament tools, no local payment, no discovery	No player dashboard, no messaging, no local payment	Too expensive, no discovery feature, no local payment	No major limitation

Table 1 Similar System Comparison

2.5.5 Critical Evaluation

When analysing the three systems, it is evident that there are guidelines to inform the design of the FootStats.

The three platforms are developed to serve Western sports markets and presuppose the payment infrastructure and organisational framework that do not fit in Nepal. The three all use Stripe or other international processors, to which most Nepali consumers and businesses lack access. FootStats is a direct response towards this by combining ESewa and Khalti, the two Nepal-dominant digital wallets.

There is a definite silver lining of community discovery. None of the three systems will enable a player to locate a local club or tournament. This indicates a design emphasis on existing administrators and not on new players aiming to join. FootStats has addressed this missing gap by providing players with a location-based search service that bridges players and clubs with those clubs and tournament organizers.

Effective solutions in this field have three design characteristics: roles separation, easy navigation, and data management. FootStats embraces all three: player, club administrator, and tournament host role-based dashboards; dynamic generation of fixtures; a react-based, tailwinds-built responsive design.

In short, the constraints of current systems and all that make up the value of foot stats: the system is specifically domain-purposed: local payment integration, community discovery, complete tournament management, and role-aware architecture all determine the ecosystem of local sports and are unique to the community.

2.6 Review Summary

Overall, the background study showed that most sports management applications in the world are aimed at professional leagues and they do not meet the needs of grassroots communities in Nepal. Current solutions generally do not feature built-in local payment support (including Esewa/Khalti), local discovery of local clubs, and easy management of amateur events. These results confirm the necessity of FootStats, a localized service that fill the gaps detected based on the community-level characteristics, local payment gateways, and reachable user interfaces.

2.7 Project risks, threats and contingency plan

All projects are associated with uncertainties that can cause a change in the development, quality, or time-prompted completion. As per the context of the FootStats project, the potential risks are reliance on the third-party payment API, potential security vulnerability of user data, the development delay caused by academic limitations, and the threat of data loss. Allowing these risks to be identified in the early stages will assist in strategizing how to prevent these risks and also it will make sure that the project can run even in cases where problems are experienced.

2.7.1 Risk and Threats

The key risks and threats of the project are delays in the implementation process because of the rigid academic schedule of the Second Year Project, technical failures when the Esewa and Khalti payment gateways were integrated, inadvertent data loss of the player statistics, and the illegal access to the system. Such threats may have adverse effects on the quality of development, user trust and the performance of the final system. Appropriate planning, periodic backups, and safe coding standards as well as constant monitoring are, thus, necessary throughout the project development.

2.7.2 Contingency Plan

To mitigate the effects of the project risks, an entire contingency plan has been drawn. Frequent data backup will also be kept to avoid losing information and the Git version control will be put to strict use to monitor the code changes and enable rollback. To mitigate the risk of technical integration, payment testing will be done in sandbox environments prior to deployment. In addition, additional buffer time is reserved in the Scrum sprints in the course of testing and debugging. Should there be dire technical challenges in any aspect in the context of development, modular development enables the team to isolate the problem without impacting the fundamental Minimum Viable Product (MVP) goals.

2.7.3 Risk Management Table

Risk	Impact	Mitigation
Data Loss	Loss of player statistics and tournament data	Keep regular database backups (PostgreSQL dumps) and use Git as version control.
Security problem	Unauthorized access to user profiles or payment data	Implement JWT authentication, bcrypt password hashing, and input sanitization to prevent SQL injection.
Failure of the API payments	The absence of the possibility to handle tournament fees	Test on the sandbox at the initial stage; introduce a set of decent error handling and manual verification fallbacks.

Table 2 Risk Management Table

2.8 Resource Requirements

FootStats project is only bound to flourish in the presence of the right hardware and software resources. These resources aid the analysis, design, code, database management, API integration, and testing processes during the project duration. Knowing them beforehand makes the implementation process a little bit easier, with the risk of avoidable delays also being minimized during the development process.

2.8.1 Hardware requirements

The hardware needed to carry out this project is a computer system with a high processing capacity, a minimum of 8GB of RAM, adequate storage space (SSD is preferable), and a good internet connection to interact with clouds. Other gadgets like smart phones and tablet computers are also needed to be used in responsive testing to make sure that the web application works properly in both desktop and mobile screens. These hardware resources make sure that development activities and real time testing can be done in an efficient manner.

2.8.2 Software requirements

This project requires modern operating systems like windows, MacOS, or Linux to use as software requirements. Frontend: The development stack is built using React.js (Vite) and Tailwind CSS, Backend: Node.js, Express, Database: PostgreSQL. Visual Studio Code is the IDE, Git and GitHub as the version control tools, and postman as the API testing tool. Moreover, they should be integrated with third-party services, such as Cloudinary to store images and Esewa/Khalti to pay. When these software resources are combined, they allow a total environment to develop and test the system.

3 Methodology

This chapter describes the methodology that was used to establish the project of foot stats in a methodical way. It explains how it was done to collect requirements, system design, implementation of key features, and testing of the end product. An articulate methodology assists in maintaining focus of the project as it is achievable and in line with the outlined goals.

3.1 Considered Methodology

A number of software development methodologies have been identified to apply to this project and this includes the Waterfall, Agile and Prototyping. The Waterfall model has definite stages in a sequence and well-documented, yet the structure is very strict and thus it is hard to incorporate changes or feedback in the process of development. Prototyping can be handy in the initial refinement of user interfaces, but it might not be structural enough to support the logic in the back-end and the database management. Agile methodology also facilitates flexibility, iterative development and continuous improvement, which is essential in a project with several features that are multifaceted such as payment integration and real-time statistics. The assessment of these approaches was used to identify the approach that suited the project scope, schedule, and nature.

3.2 Chosen Methodology

Following the assessment of various strategies, the Scrum framework (Agile methodology) was chosen in this project since it is the most suitable in the face of the development requirements and academic schedule. According to the project proposal, Scrum enables the team to divide complex tasks in small and manageable sprints of 1-2 weeks. This methodology supports continuous improvement with the use of stand-up meetings on a daily basis, sprint reviews, and retrospectives, so that the team will be able to adjust to challenges and implement feedback as fast as possible (Muslihat, 2018). It offers a reasonable trade-off between design, quality management, and efficiency of implementation, and it is therefore suitable to implement a working Minimum Viable Product (MVP) within the Second Year Project timeline.

4 System Design

System design is a step of software development that entails fine-grained planning that involves a structured blueprint of the whole system prior to the actual start of coding. It is much more than just defining components. It involves making the appropriate choice of architecture (layered or MVC), designing databases, specifying the way various components of the system interact, and making sure the system is capable of supporting future expansions (scalability) and high-performance. Security, error handling and user experience are also factored in the system design in an effort to ensure that the system is reliable and efficient. It is typically categorized into high-level design (system architecture in general) and low-level design (logical details of the single components). A properly designed system simplifies the development process, minimizes mistakes, and leads to final product being developed and executed to satisfy user and business requirements (Singh, 2025).

4.1 System Architecture

System architecture is the top-level design of a system specifying the structure of the components of the system and how these interrelate with one another. In order to make the system efficient and satisfy the user's needs, it describes the general design, hardware and software, databases and their interconnections. System architecture is used as a guideline giving the developers a guide on how to develop and incorporate various components of the system taking into consideration the various factors such as scalability, performance, security and maintainability. It assists in the analysis of data flow of the system and the manner in which various modules interact in order to generate desired functionality (Tiwari, 2025).

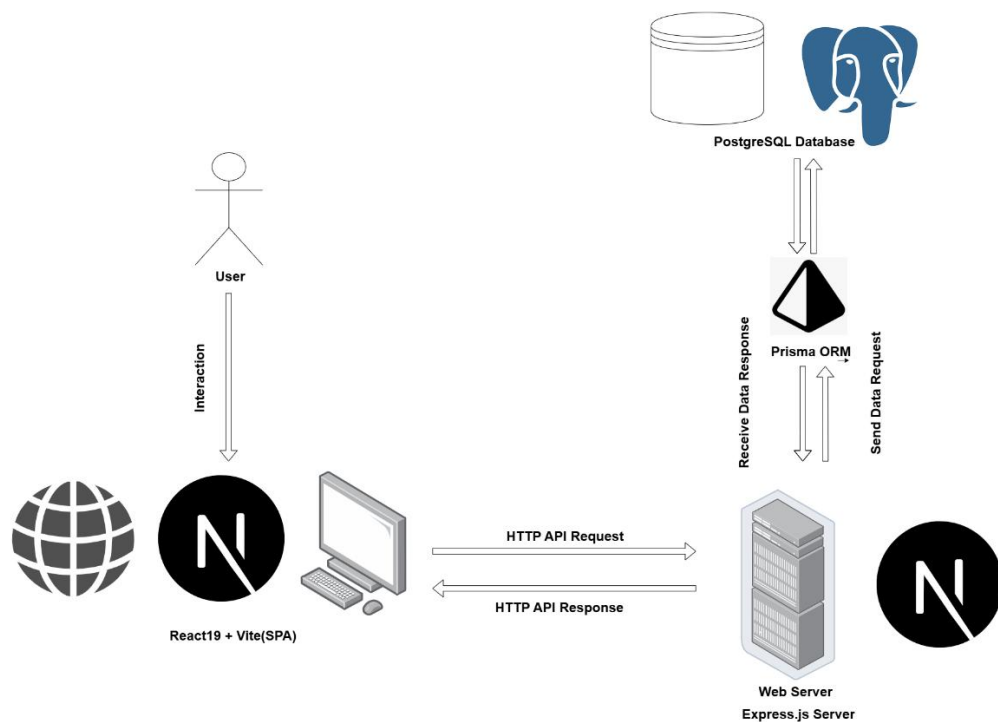


Figure 2 System Architecture

The suggested system follows a three-tier architecture consisting of presentation, application and data layers. The presentation layer is created based on React with vite (SPA) and through the responsive web interface, the users would be engaging with the system. An Express.js web server handles the application layer and processes the HTTP API requests and integrates business logic, and handles the communication between the frontend and the backend. The data layer is based on PostgreSQL database, where the Prisma ORM is an intermediate to facilitate data queries, storage, and retrieval in an effective manner. The architecture is designed in such a way that there is clear separation of concerns, enhanced maintenance as well as the ability to scale efficiently as the number of users increases.

4.2 Entity Relationship Diagram (ERD)

Entity Relationship Diagram is a diagram in the database design which is used to show how data is organized and their relationship. It uses shapes such as rectangle for entities, oval for attributes. It shows relationship between each entity by using crow's foot notation. It makes the complex data requirements into a simple diagram which helps the designers to know how the system flows in the data (Belic, 2025).

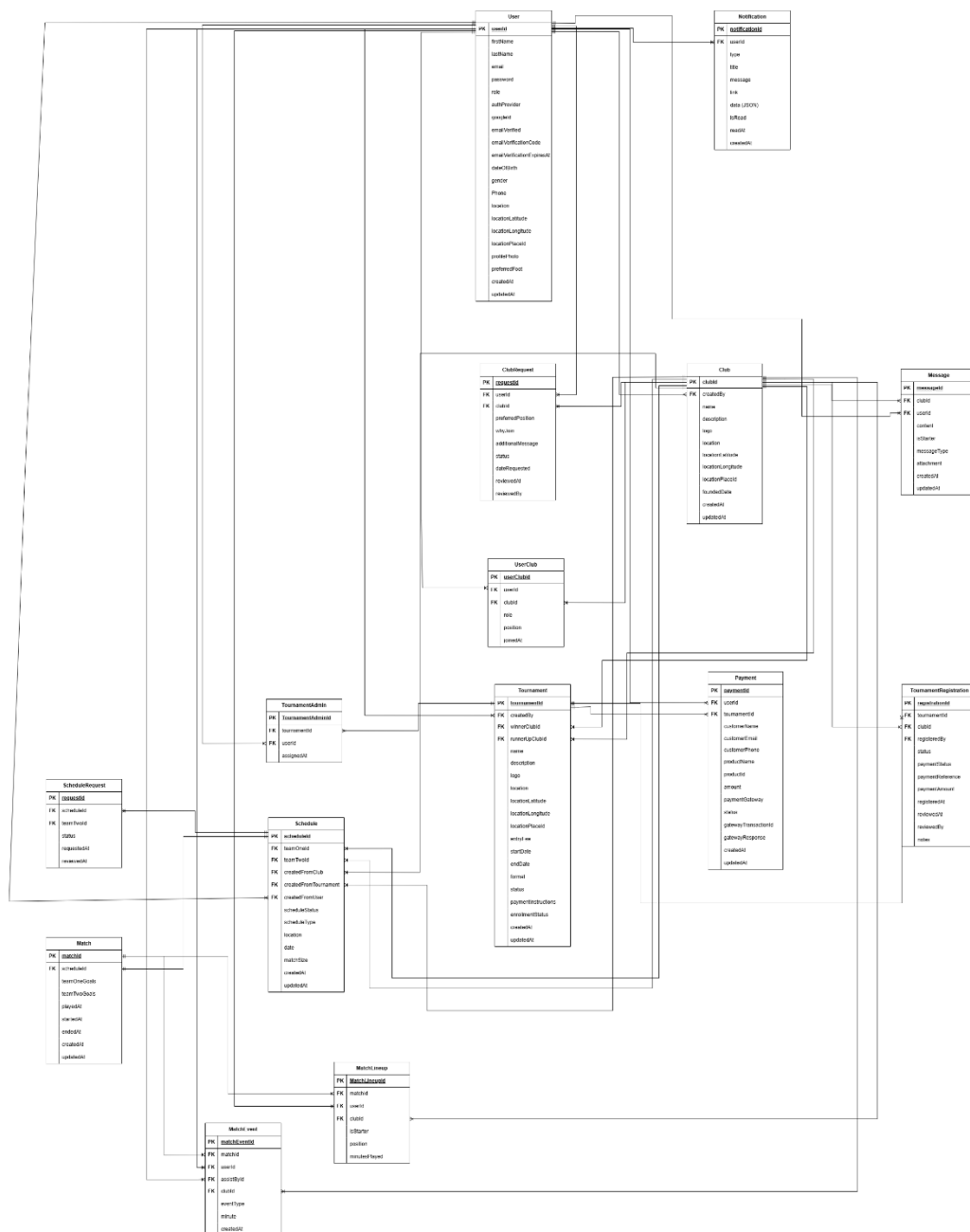


Figure 3 ERD

The ERD is a sports/tournament management system in which the User entity is the central one which contains personal information and is connected to virtually all other entities. The users may form or join Clubs their membership is governed by the UserClub table, and the join requests are processed by ClubRequest. The Tournaments can be done in each of the clubs, and they may contain such information as name, place, entry fee, and timing. On TournamentRegistration, users are allowed to register to tournaments and such payments are made on the Payment entity. Tournament matches are organized in Schedule/Match and the participation of a player in a match is tracked in detail using MatchLineup and events like goals or actions on a match are stored in the MatchEvent. Message (club chats) and Notification (user alerts) are the communication features that it supports. Also, administration is done through TournamentAdmin and scheduling requests or issues are done through ScheduleRequest. In general, ERD indicates a very interrelated system that controls users, clubs, tournaments, matches, communication, and payments in structured relational system.

4.3 Use Case Diagram

Use case diagram is a UML (Unified Modelling Language) diagram, which represents visually the communication between various users known as actors to a system to accomplish a set of goals. It illustrates the system as a boundary, and it contains different use cases (functions or services) that are offered by the system, e.g. logging in, making a payment, or managing data. People, organizations and other systems that interact with the system can act as actors. The diagram assists in interpreting the functional requirements of a system in a simple way that clearly demonstrates what the system does in the eyes of the user without emphasizing on how these functions are carried out (Contributor, 2020).

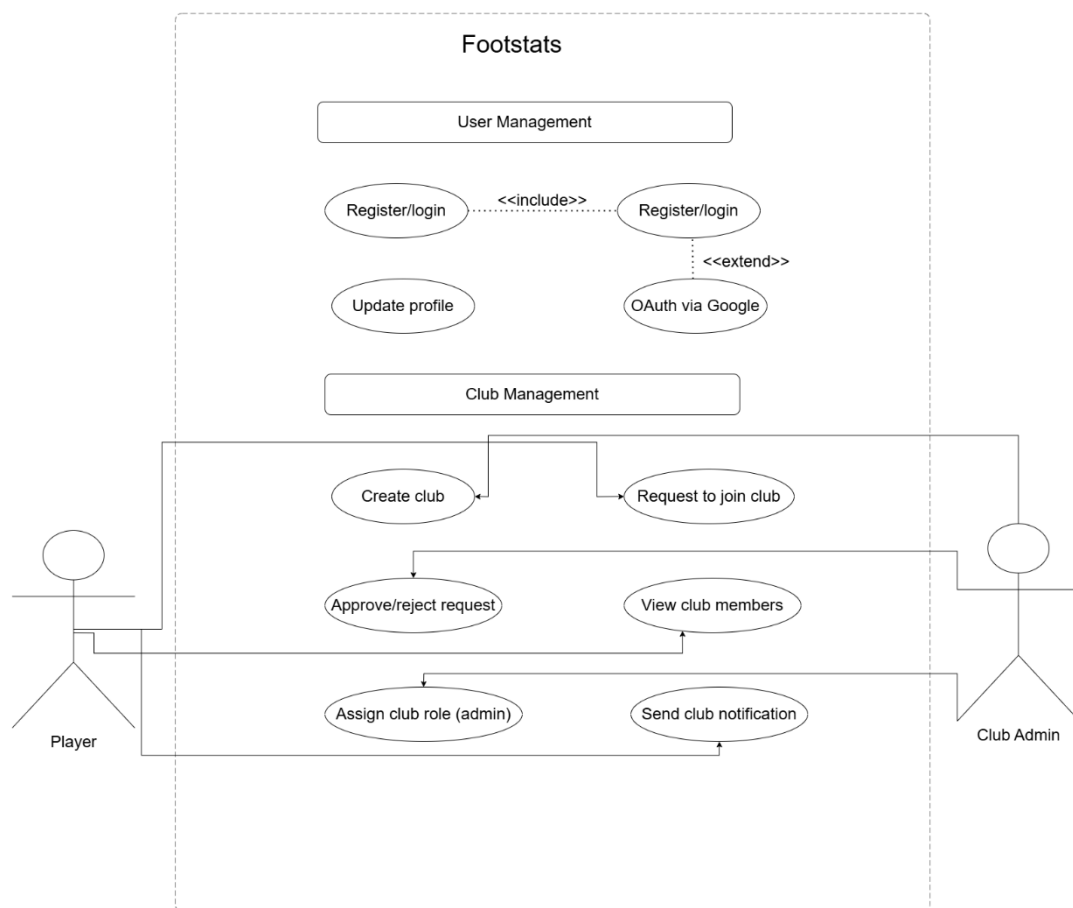


Figure 4 Use case diagram

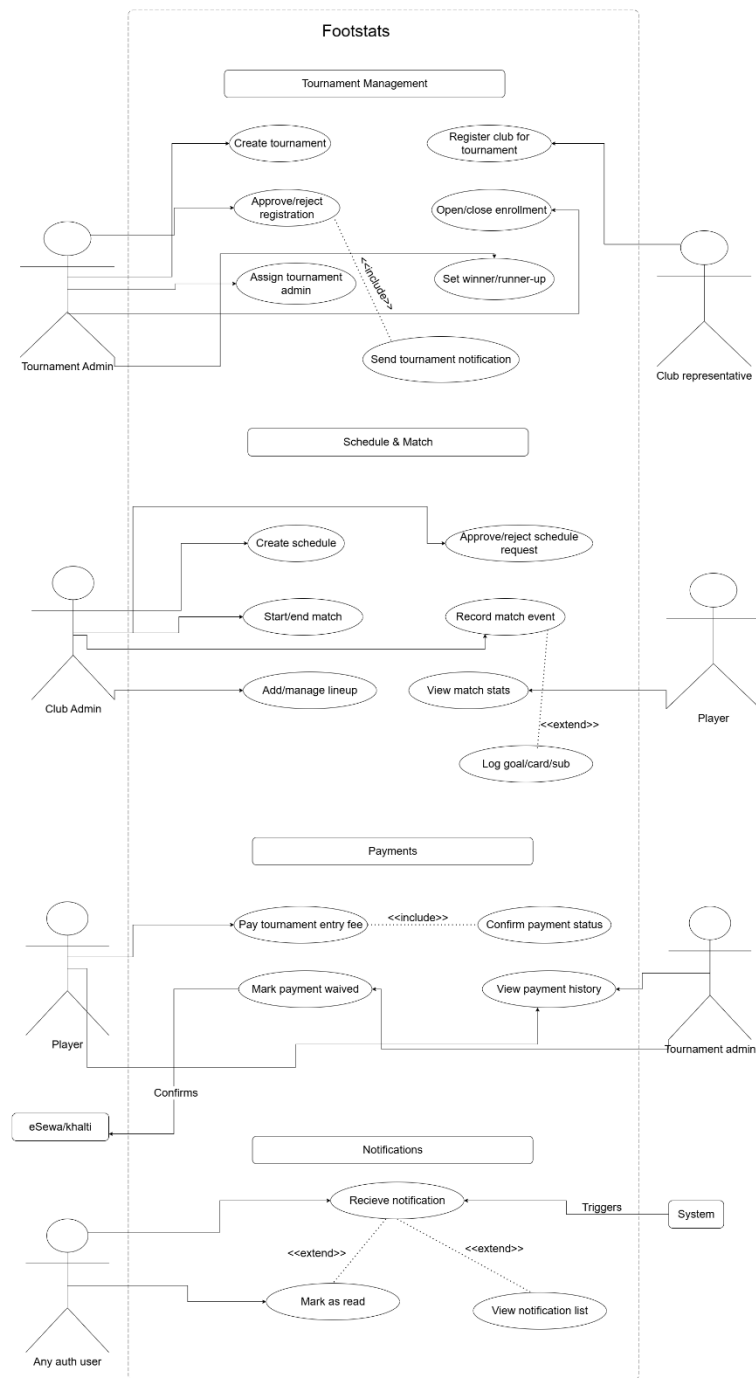


Figure 5 Use case diagram

The following Use Case Diagram demonstrates the interaction of different users with the FootStats system through different functional modules. It has several actors like player, club admin, tournament admin and general authenticated users whose role and permissions differ. The system will be split into the main modules such as User Management, Club Management, Tournament Management, Schedule and match, Payments and Notifications. Under User Management, users are able to add/log in (OAuth is also available through Google) and renew their profiles. The Club Management also allows players to request membership in clubs and club admins to create clubs, accept or decline requests, give assignments, and issue notifications. Tournament Management enables tournament admins to create tournaments, handle registrations, assign admins and declare victors whereas club representatives can also enter tournaments and handle their participation. The Schedule & Match module deals with the creation of matches, their approval, the registration of their events, and the observation of the results, and players add match events such as goals or substitutions. Payment's module allows individuals to determine the payment of fees during tournaments, check payment status, and history with external services, ESewa/Khalti. Lastly, the Notifications module helps users to be informed about the updates in the system, see the notifications and mark them as being read. Such use case organization clearly establishes how the system will be used, the roles played by users, and their interactions thereby providing a well-organized and smooth system behaviour.

4.4 Flowchart

A flowchart is a pictorial illustration of a procedure or algorithm that illustrates the order of tasks that must be followed to accomplish a task. It also employs conventional symbols like ovals (start/end), rectangles (process) and diamonds (decision) and arrows (flow lines) to depict the relationship between actions and decisions. Flowcharts are beneficial in simplifying the complex processes as they can be broken down into understandable, logical steps and one can have a better understanding, analysis and communication on how the system or procedure works. They find much application in both programming and system design and problem-solving to design and visualize workflow before implementation (Scheuer, 2025).

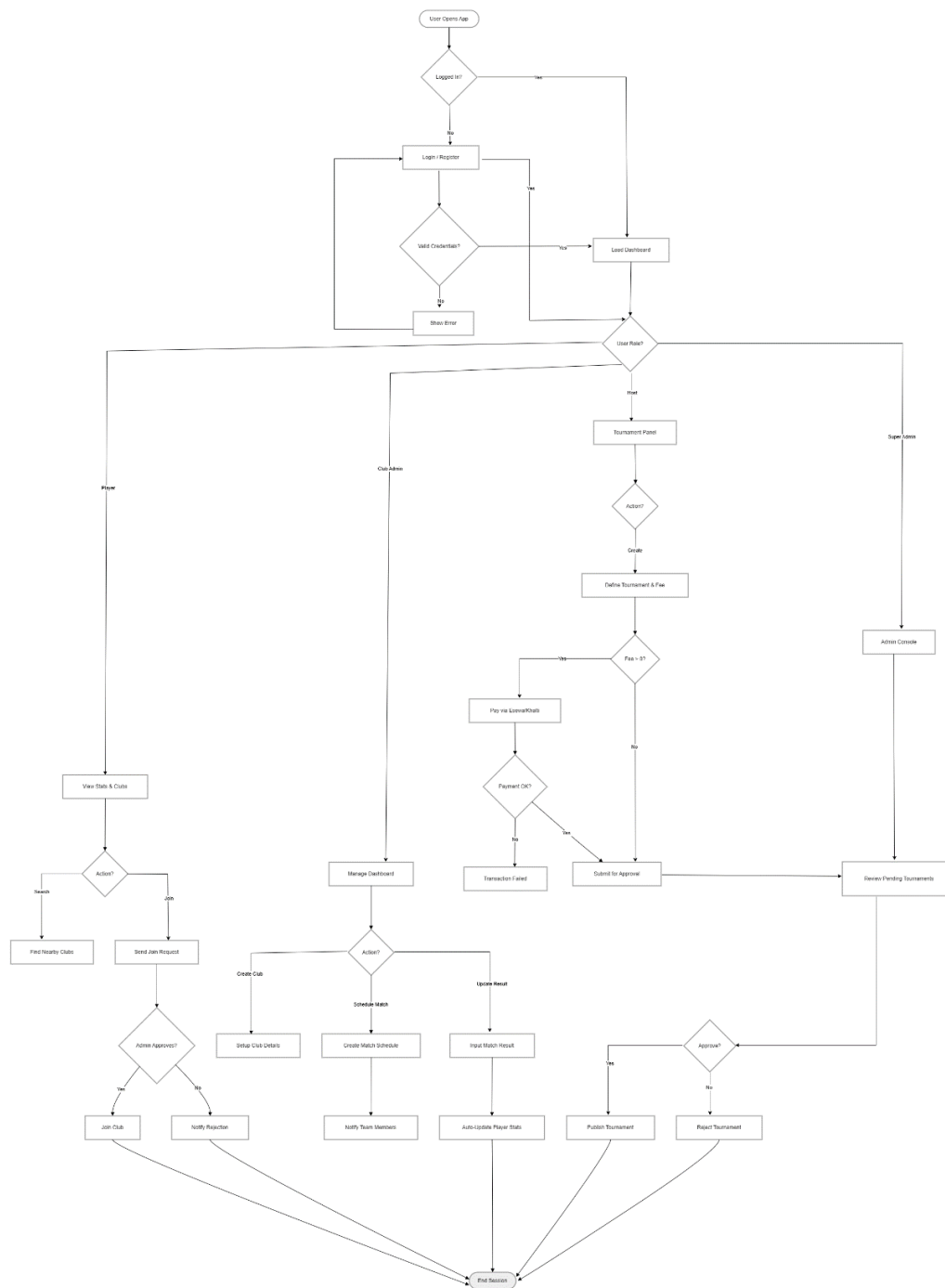


Figure 6 Flowchart

The following is a flowchart illustration of the system workflow of a sports/tournament management application, which demonstrates the interplay of various components in response to user activities down to the processing in the back-end. It starts with the user (player, club owner, or tournament administrator) issuing commands such as authentication (sign up/login) and delivers those via the client layer and processes them via API calls. The system directs requests to validation, authorization middleware and controllers depending on the actions of the user creating clubs, scheduling matches, organizing tournaments, sending messages or making payments. These controllers do logic processing and communicate with the database (storing in data such as user, club, match, schedules, messages, and notifications). It also has decision points in the flow to deal with errors, request validation and payment verification. Lastly, the results are displayed to the user by displaying processed data back to the frontend via APIs. The diagram at large shows a whole end-to-end flow of user input to the application, processing, storage, and output in the structured full-stack application.

5 Software Requirements Specification (SRS)

The SRS chapter outlines the specifications of the proposed FootStats system down to the structured and quantifiable manner. It defines the functional behaviour that the system should have such as register a player, track his or her statistics, and integrate payments with the system as well as the quality attributes that the system is supposed to maintain which include the performance, security, and reliability. This documentation assists the developers, evaluators and users to know the precise scope and expectations of the project to act as a point of reference during design, implementation and testing (Sigdel, 2025).

5.1 Functional Requirements

The functional requirements explain the kind of tasks that the FootStats is expected to carry out in order to satisfy the players, club administrators, tournament hosts and super administrators. These are requirements that encompass all phases of grassroots sports management such as user authentication and authorization, profile and statistics management, club creation and membership processes, tournament organization and registration, match scheduling and result tracking, payment processing and transaction management, real time communication capabilities, and administrative oversight capabilities. Every requirement states clear anticipated behaviour that has quantifiable acceptance criteria allowing systematic implementation and validation by testing.

ID	Requirement Statement
FR-01	The system shall allow user to register and log in.
FR-02	The system will show the player profile and match participation will be recorded automatically.
FR-03	The system shall calculate and update statistics such as goals, assists and win based on match results.
FR-04	It will display ranking leader boards and a club-by-club basis relating to player performance.
FR-05	The system will permit Club Admins to add, remove, or change the details of a club and add / remove players to it.
FR-06	Players shall be able to request join to clubs through the system, which will be approved or rejected by Club Admins.
FR-07	The system will permit admins to schedule matches with date, time, and location and avoid conflicts on the schedule.
FR-08	The system will provide notifications on match schedules and updates to the parties involved.

FR-09	The system will enable club admins to enter matches results, which automatically update player statistics.
FR-10	The system will enable Tournament Hosts to establish tournaments fee and other description about tournament.
FR-11	The system will accept online payments through a combination of Esewa and Khalti to register in the tournament.
FR-12	The system will not confirm a team as registered in a tournament without verifying the success of payment or the host accepts the request without payment.
FR-13	The system will offer internal chat system for a club to make communication between the members of the club.
FR14	The sites will show clubs and tournaments around the user based on his/her location.

Table 3 Functional Requirement

5.2 Non- functional Requirements

The non-functional requirements lay down the quality requirements that FootStats will have to meet outside of its functional capabilities. These requirements deal with performance of the system, security of system, considerations of usability, reliability expectations, scalability capabilities, maintainability considerations, compatibility requirements and availability guarantee. Non-functional requirements unlike functional requirements explains how the system performs its functions well. These quality criteria are important since they have a direct effect on the quality of the system as a whole, the level of user satisfaction, adoption, and the long-term viability of the platform.

Category	Requirement Statement
Performance	The system should also be loaded with the homepage within 2 seconds and be able to support up to 1,000 simultaneous users.
Security	JWT should be used to authenticate the system, password hashing should be done with bcrypt, and input sanitization should be used to avoid SQL and XSS attacks.
Reliability	The system is expected to have 99% uptime and records of the tournament must never be lost.
Interaction Usability	The interface must be usable on desktop, tablet, and mobile devices and must have a clean sports-themed GUI and dark/green themes.
Maintainability	A modular structure of the backend (Node.js/Express) must be used to enable easy additions and updating the system.
Scalability	The system must be made to support increased amounts of users in the future.

Table 4 Non-Functional Requirements

6 Testing

6.1 Authentication (Register)

Field	Description
Test Case ID	TC_AUTH_01
Module	Registration
Input	Valid username, email, password
Expected Result	Successfully created user account
Actual Result	Account created successfully
Status	Pass

Table 5 Test case of registration

In this test case, it is confirmed that a new user can register successfully using a valid user name, email and password. The new account should be created and the input accepted by the system. A successful registration is an indication of correct validation and storage of data.

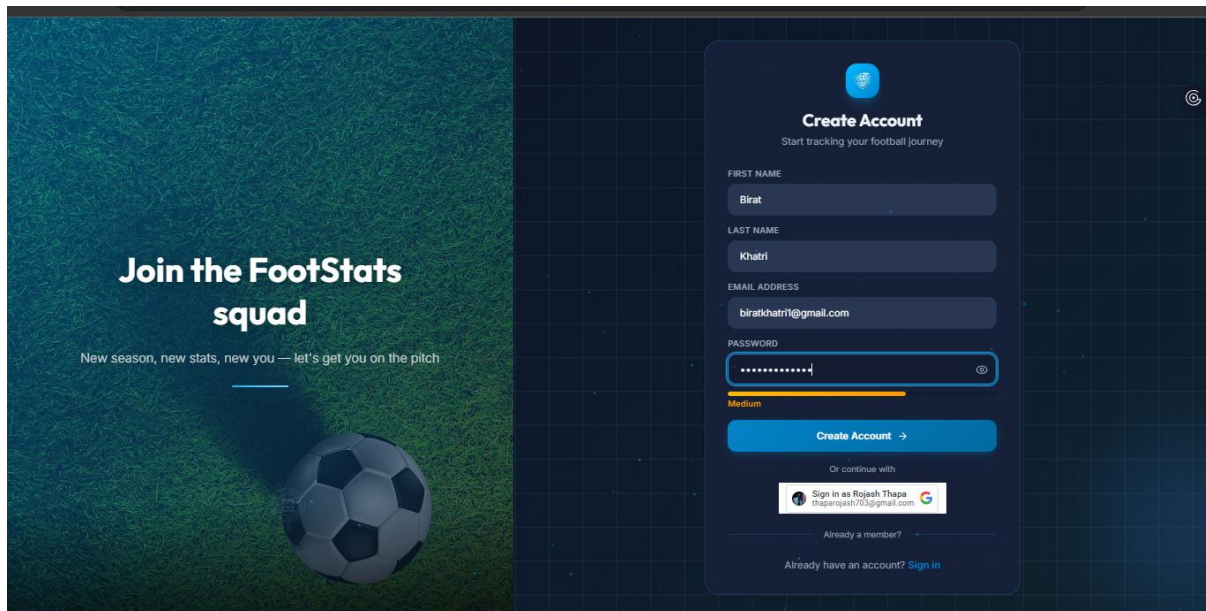


Figure 7 Register User Test

6.2 Authentication(Login)

Field	Description
Test Case ID	TC_AUTH_02
Module	Login
Input	Correct email and password
Expected Result	User is redirected to dashboard
Actual Result	Dashboard displayed successfully
Status	Pass

Table 6 Test case of Login

In this test case, the user is tested in terms of his/her ability to log in with proper criteria. The system is supposed to authenticate the user and redirect him/her to the dashboard. This makes sure that the login functionality is functioning properly.

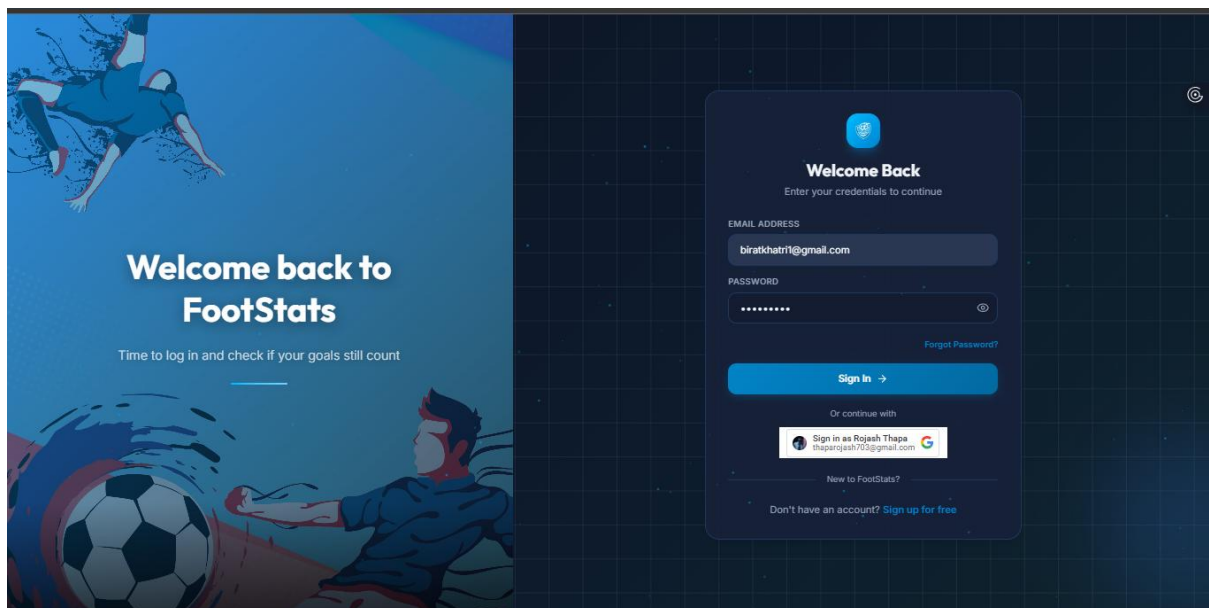


Figure 8 Login Test

6.3 Club Creation

Field	Description
Test Case ID	TC_CLUB_01
Module	Club Creation
Input	Valid club name and details
Expected Result	Successfully created club
Actual Result	Club created successfully
Status	Pass

Table 7 Test case of club creation

This test case is to make sure that a user is capable of creating a club by supplying sound club information. The information is to be stored in the system and confirmed. This confirms adequate management of club creation functionality.

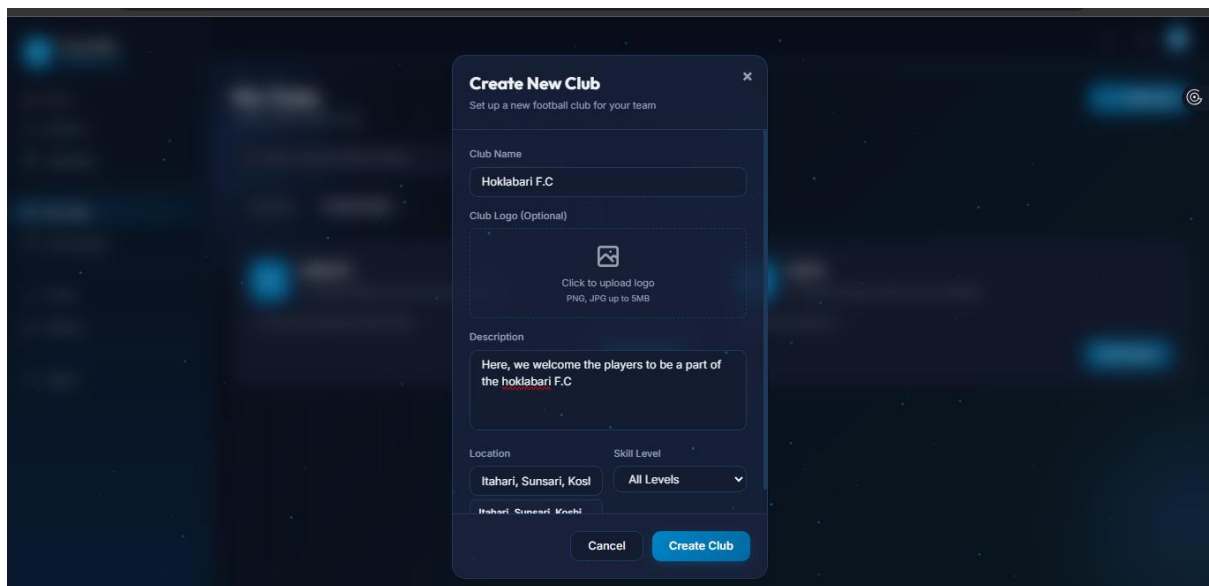


Figure 9 Creating Club

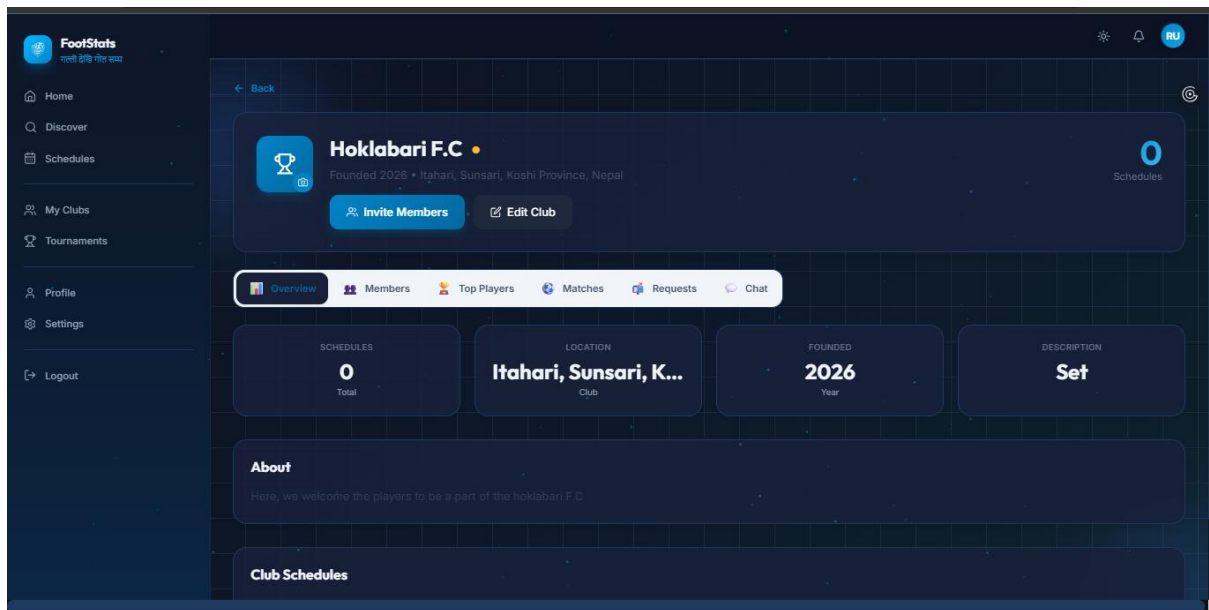


Figure 10 Club Created

6.4 Tournament Creation

Field	Description
Test Case ID	TC_TOUR_01
Module	Tournament Creation
Input	Valid tournament details
Expected Result	Successfully created tournament
Actual Result	Tournament created successfully
Status	Pass

Table 8 Test case of Tournament Creation

This evaluation case confirms that it is possible to prepare a tournament using the appropriate input information. The system is supposed to run the data and be effective to form the tournament. This proves that the module is working as anticipated.

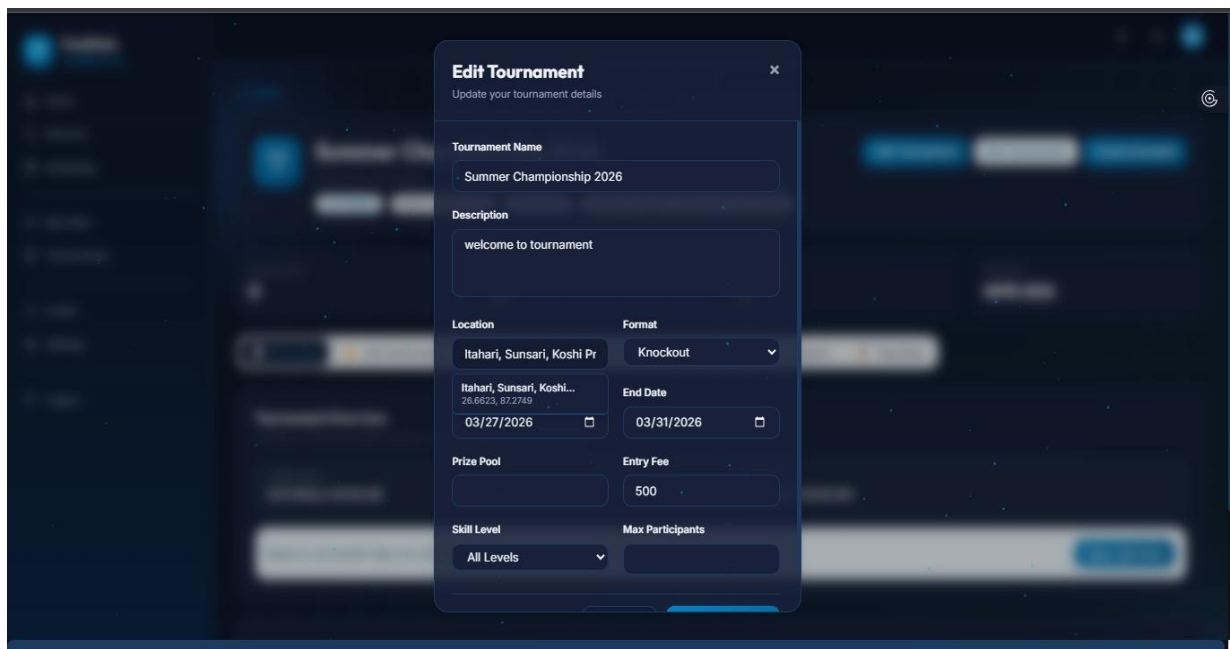


Figure 11 Creating Tournament

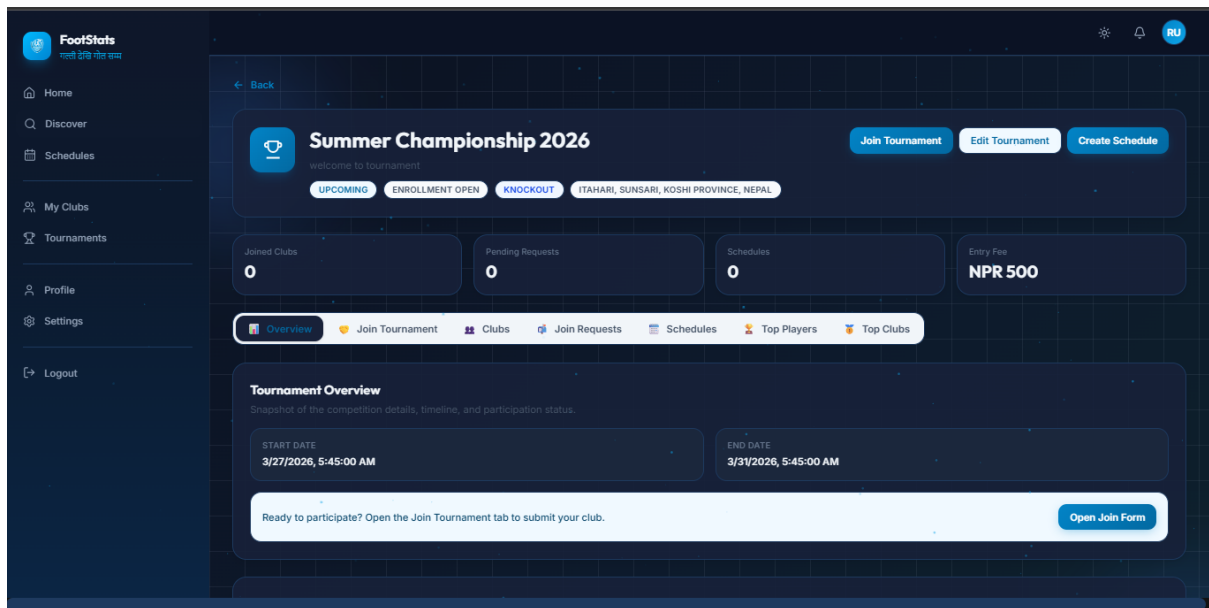


Figure 12 Tournament Created

6.5 Challenge Club

Field	Description
Test Case ID	TC_SCH_01
Module	Challenge club
Input	Select opponent club and send challenge
Expected Result	Successfully sent challenge request
Actual Result	Challenge sent successfully
Status	Pass

Table 9 Test case of Challenge Club

This test case is used to examine the ability of a club to issue a challenge to another club. The challenge request should be generated and sent successfully by the system. This is to ensure that there is appropriate interaction between clubs.

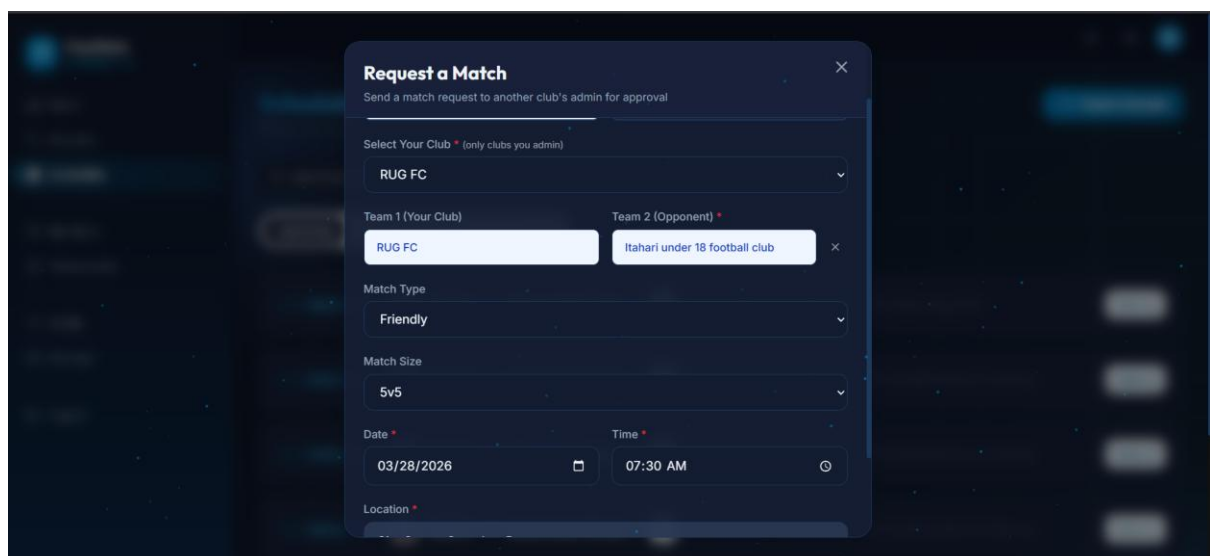


Figure 13 Challenging the club during schedule creation

6.6 Accept Challenge & Schedule Match

Field	Description
Test Case ID	TC_SCH_02
Module	Scheduling
Input	Accept challenge request
Expected Result	Successfully scheduled match
Actual Result	Match scheduled successfully
Status	Pass

Table 10 Test case of Accept challenge and schedule match

This test case confirms the fact that challenge acceptance incurs match scheduling. The system will have to correct the match date and validate the schedule. This ensures proper scheduling capability.

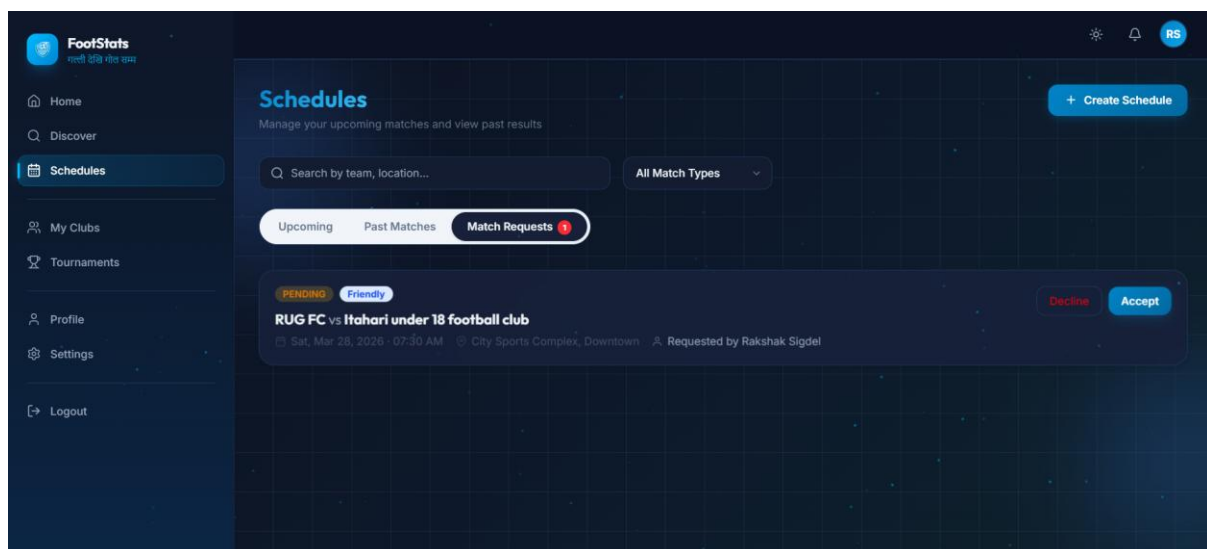


Figure 14 Match request is received by another club admin

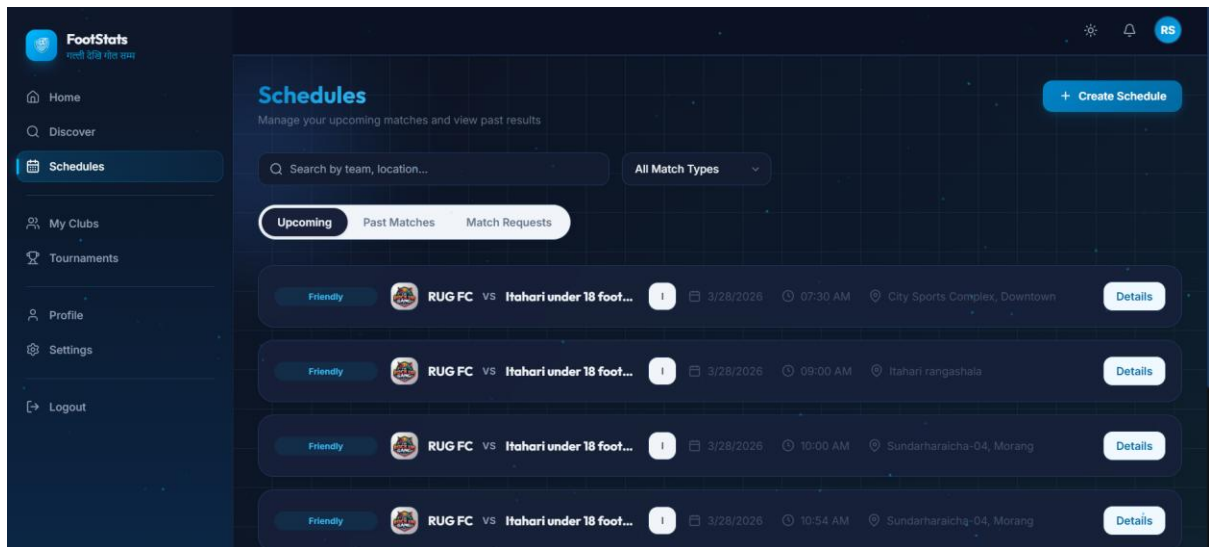


Figure 15 Matches are scheduled

6.7 Club Chat System

Field	Description
Test Case ID	TC_CHAT_01
Module	Chat System
Input	Send valid message
Expected Result	Successfully delivered message to club members
Actual Result	Message displayed instantly
Status	Pass

Table 11 Test case of Club Chat System

This test case will make sure that users are able to send messages in the club chat system. The system needs to provide real-time messages to the other members. This justifies the communicating aspect.

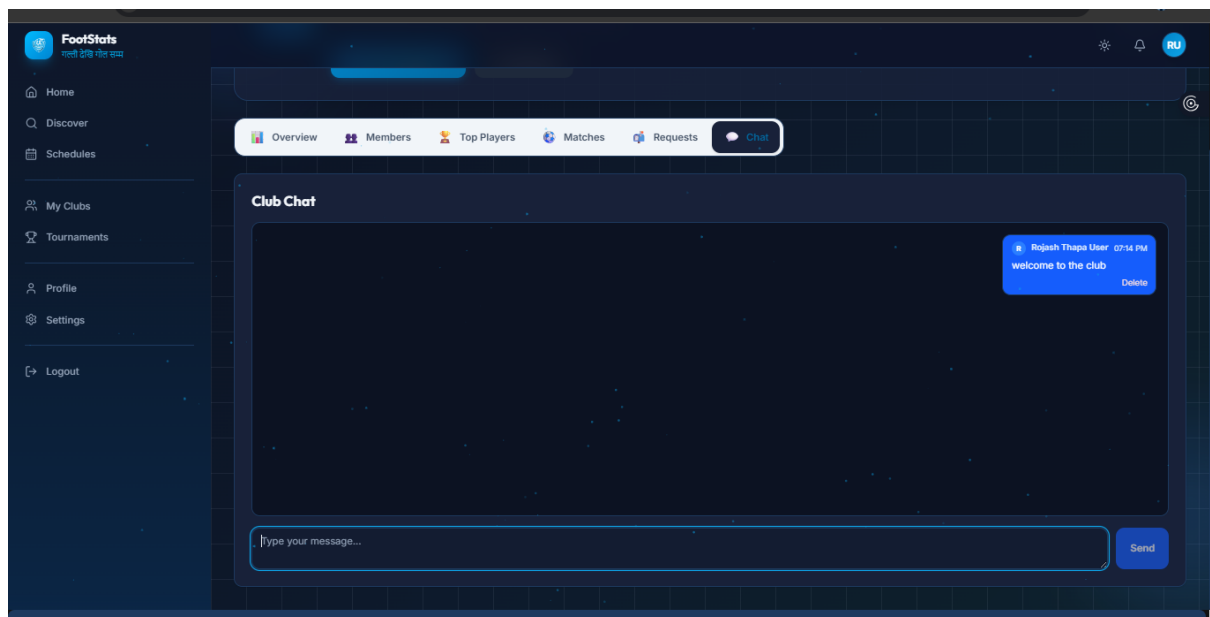


Figure 16 Chat System test

6.8 Match Event Recording

Field	Description
Test Case ID	TC_MATCH_01
Module	Match Events
Input	Add goal event
Expected Result	Successfully recorded event
Actual Result	Goal event added successfully
Status	Pass

Table 12 Test case of Match Event Recording

This test case will make sure that users are able to send messages in the club chat system. The system needs to provide real-time messages to the other members. This justifies the communicating aspect.

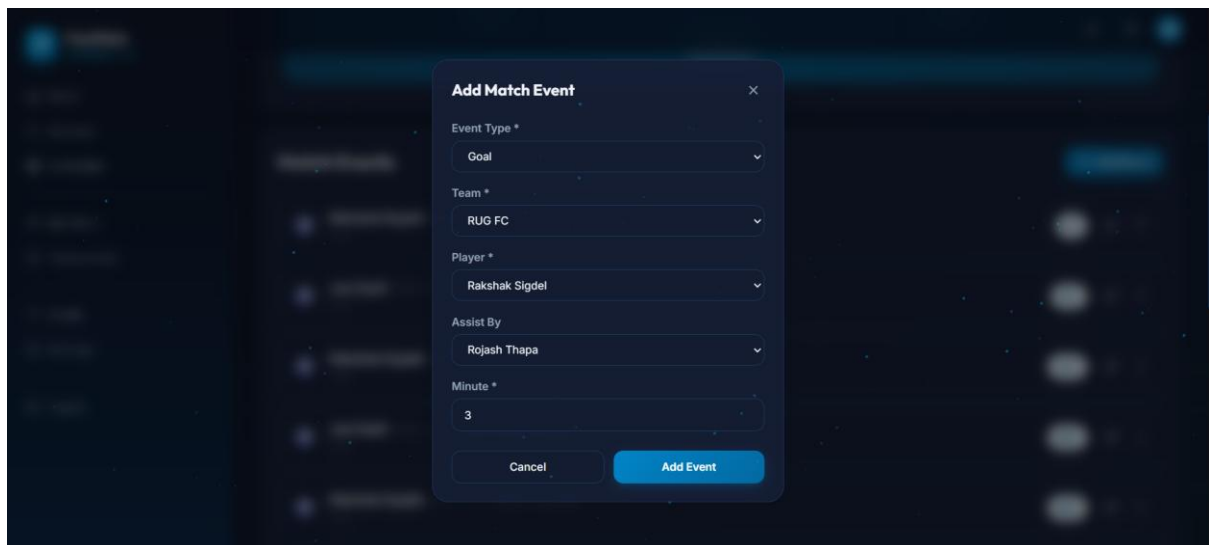


Figure 17 Add match event

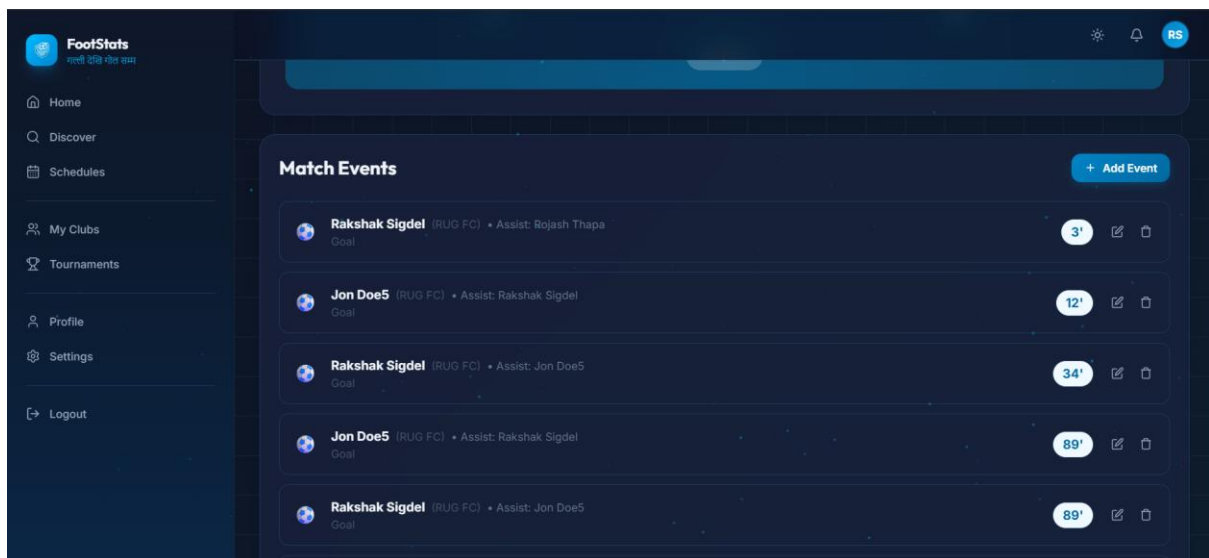


Figure 18 Match event added successfully

6.9 Notification System

Field	Description
Test Case ID	TC_NOTIF_01
Module	Notification
Input	Trigger event
Expected Result	Successfully sent notification to users
Actual Result	Notification received successfully
Status	Pass

Table 13 Test case of Notification System

This test case will make sure that users are able to send messages in the club chat system. The system needs to provide real-time messages to the other members. This justifies the communicating aspect.

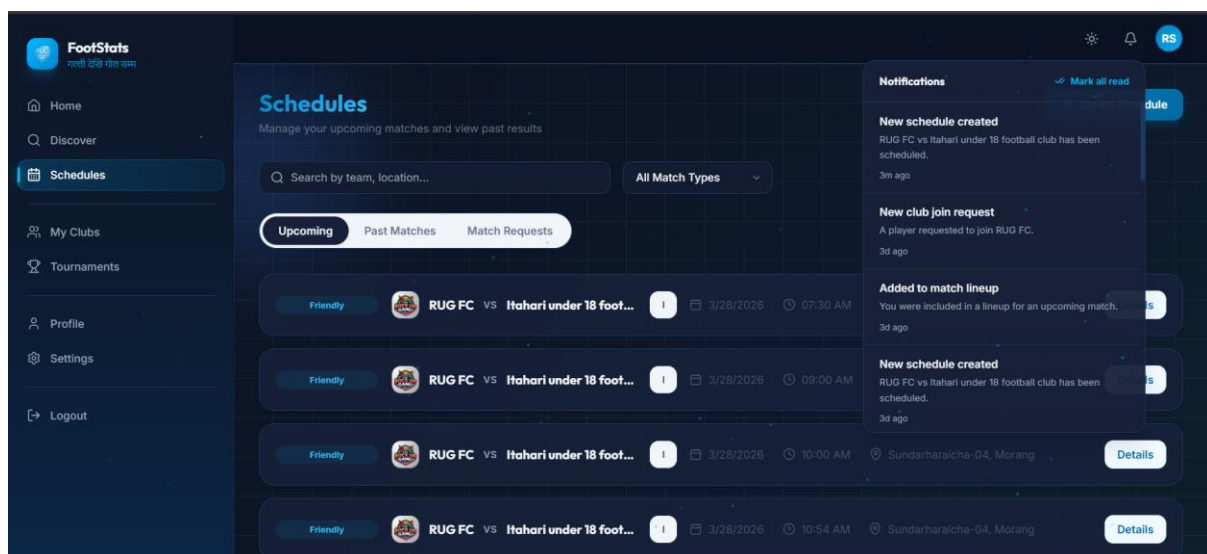


Figure 19 Notification for new schedule



Figure 20 Notification through email

7 Group Contribution Table

The project was undertaken under a group effort and every member worked in various stages involved in the project like research, analysis, design, development, testing, and documentation. The distribution of duties based on the personal strengths and delegation of tasks was provided; however, everyone contributed to the discussion, decision-making, and completion of the project. The contribution details presented below are the contribution and participation of each member in the successful implementation of the project: FootStats.

Member Name	London Met ID	Major Responsibility	Key Responsibility
Rojesh Thapa	24046023	Backend Features and Integration Lead	Rojash implemented backend features at the service and controller level. He documented how modules connect with APIs and data flow, then validated API behavior during integration and checked that critical modules worked reliably after deployment.
Rakshak Sigdel	240460001	Project Coordination, Deployment, and Cross-Team Delivery Lead	Rakshak connected frontend and backend efforts and kept the team aligned on delivery goals. He maintained shared standards across documentation and design, oversaw integration readiness,

			and coordinated final validation to ensure reliable releases.
Samana Upreti	24046039	Frontend Experience and Documentation Lead	Samana built and refined key user flows, keeping the interface clean and consistent. She documented frontend features and interaction behavior, then validated UI builds and tested page-level functionality across common user scenarios before releases.
Bibek Chaudhary	24045843	Frontend Architecture and Quality Lead	Bibek designed the frontend structure and component architecture, focusing on navigation and maintainability. He documented component updates and page logic, then verified layouts and functionality across different views to ensure stable production behavior.

Table 14 Contribution Table

8 Conclusion

In the conclusion, the project was developed to address the problem of unorganized management, lack of statistics tracking and poor communication within the amateur football and futsal communities in Nepal with the development of FootStats web-based application. This project was successfully analysed the needs of user, designed the required systems components using modern web technologies and also defined the important features like club creation, tournament registration, scheduling, notification and payment integration. Through the implementation of the platform, the work demonstrates that the proposed system can significantly improve the coordination and user engagement within the local sports domain and grassroot of this industry or platform. The FootStats met its primarily objectives of transforming traditional, manual process with a digital platform which further provides a strong foundation for grassroots sports management.

8.1 Legal Issues

The project has to meet the need to address the legal responsibility related to software use, data protection and other intellectual property. FootStats gathers personal data (football player names, contact information, success statistics and so on) so it is essential to handle such data in a legal and responsible manner. The system complies with data protection principles by ensuring that user data is collected with consent and used if for the purpose of sports management. Along with that the integration of payment gateways like Esewa and Khalti requires strict compliance with their terms of service and Nepali financial regulation concerning digital transactions. The project guarantees the use of open-source software libraries and tools that are adequately licensed so as to prevent copyright violation.

8.2 Social Issues

The system has various social context because it promotes community engagement and a healthier lifestyle by making sports organization more accessible. FootStats helps provide organizations with the next layer of visibility (footballs to player, club, and organizer), regardless of how small a participant in the sport might be. Digitizing the process reduces the barriers to entry into holding tournaments, which may raise sports participation rates. However, the developers must consider the digital divide should also be taken into account, the interface has been designed to be mobile friendly and user friendly which helps the users with limited access to computers or those with limited technical knowledge can still participate equally in the sporting community.

8.3 Ethical Issues

The use and creation of the FootStats system are ethically-oriented. All data about users, especially performance, should be treated with the highest level of confidentiality. The system also uses role-based access control that ensure the sensitive data is only accessible to a authorized user. A transparent record of fees obtained with the help of digital payment can also assist in building trust between the organizers and participants of sporting tournaments by excluding the possibilities of financial maleficence.

8.4 Future Work

Despite the fact that the present project achieves the overall goals of developing a working web-based management system, a number of suggestions can be addressed in the future work. These involve creation of native mobile apps based on Android and iOS so that users can have better accessibility on the move. In the future improvement we could integrate Artificial Intelligence (AI) to provide auto fixtures generation, personalized training recommendations for players and advance analytics for club performance. Besides, extending the scope to cover other sports and incorporation with social media platforms to reach more individuals would make the project useful and more effective in the long-term.

9 References

- 360Player. (2024) *360player* [Online]. Available from: <https://www.360player.com> [Accessed 03 March 2026].
- Belic, I. (2025) *IBM* [Online]. Available from: <https://www.ibm.com/think/topics/entity-relationship-diagram> [Accessed 03 March 2026].
- Contributor, T. (2020) *TechTarget* [Online]. Available from: <https://www.techtarget.com/whatis/definition/use-case-diagram> [Accessed 03 March 2026].
- LeagueRepublic. (2024) *leaguerepublic* [Online]. Available from: <https://www.leaguerepublic.com> [Accessed 03 March 2026].
- Muslihat, D. (2018) *Agile methodology: an overview* [Online]. Available from: <https://medium.com/zenkit/agile-methodology-an-overview-7c7e3b398c3d> [Accessed 25 March 2026].
- Scheuer, K. (2025) *Mural* [Online]. Available from: <https://www.mural.co/blog/flowcharts> [Accessed 03 March 2026].
- Sigdel, R. (2025) *Software Requirement Specification*. Acedemic Report. Itahari: Itahari International College.
- Singh, A.P. (2025) *AlgoMaster* [Online]. Available from: <https://algomaster.io/learn/system-design/what-is-system-design> [Accessed 03 March 2026].
- SportEasy. (2024) *Sporteasy* [Online]. Available from: <https://www.sporteasy.net> [Accessed 03 March 2026].
- Thapa, R. (2025) *Proposal*. Acedemic Report. Itahari: Rakshak Sigdel Itahari Internationa College.
- Tiwari, V. (2025) *Medium* [Online]. Available from: <https://medium.com/@vaibhaviwari.945/understanding-system-architecture-a-beginners-guide-d2aad48f0007> [Accessed 03 March 2026].

10 Appendix

The appendix will contain the supplementary materials that will support the project of FootStats but will be excluded in the main discussion to provide clarity and flow. These sources give additional evidence on project planning, requirement gathering, technical specifications, and project management artifacts without discontinuing the major part of the report.

10.1 Code Snippets

This section includes the important Part of the code in the project.

```
import paymentRoutes from "../routes/paymentRoutes.js";
import locationRoutes from "../routes/locationRoutes.js";

const app = express();
app.use(cors());
app.use(express.json());

// Serve static files from uploads directory
app.use('/uploads', express.static(path.join(__dirname, '../uploads')));

//Routes
app.use("/api/auth", authRoutes);
app.use("/api/players", playerRoutes);
app.use("/api/tournaments", tournamentRoutes);
app.use("/api/clubs", clubRoutes);
app.use("/api/schedules", scheduleRoutes);
app.use("/api/matches", matchRoutes);
app.use("/api/requests", requestRoutes);
app.use("/api/schedule-requests", scheduleRequestRoutes);
app.use("/api/match-events", matchEventRoutes);
app.use("/api/match-lineups", matchLineupRoutes);
app.use("/api/messages", messageRoutes);
app.use("/api/notifications", notificationRoutes);
app.use("/api", paymentRoutes);
app.use("/api/locations", locationRoutes);
app.get("/", (req, res) => {
  res.send("Welcome to Footstat API");
});

const PORT = process.env.PORT || 5555;
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Figure 21 Backend Entry Point and Route Wiring

```

Backend > prisma > schema.prisma > ...
1 // This is your Prisma schema file,
2 // learn more about it in the docs: https://pris.ly/d/prisma-schema
3
4 Generate
5 generator client {
6   provider = "prisma-client-js"
7 }
8
9 datasource db {
10   provider = "postgresql"
11   // url      = env("DATABASE_URL")
12   // directUrl = env("DIRECT_URL") // Optional: for migrations
13 }
14
15 // =====
16 // USER MODEL
17 // =====
18 model User {
19   id          Int    @id @default(autoincrement())
20   firstName   String
21   lastName    String
22   email       String @unique
23   password    String?
24   role        Role   @default(PLAYER)
25   authProvider AuthProvider @default(LOCAL)
26   googleId    String? @unique
27   emailVerified Boolean @default(false)
28   emailVerificationCode String?
29   emailVerificationExpiresAt DateTime?
30
31   dateOfBirth DateTime?
32   gender        String?
33   Phone         String?
34   location      String?
35   locationLatitude Float?
36   locationLongitude Float?
37 }

```

Figure 22 Prisma Schema

```

import AuthService from "../services/authService.js"
export const register = async (req, res, next) => {
  try {
    const { firstName, lastName, email, password } = req.body;
    if (!firstName || !lastName || !email || !password) {
      return res
        .status(400)
        .json({ message: "Please fill all the required fields!!!" });
    }
    const user= await AuthService.register({
      firstName,
      lastName,
      email,
      password,
    });
    res
      .status(201)
      .json({message: "Registered successfully. You can verify your email later in s
    } catch (error) {
      res
        .status(error.status || 500)
        .json({ message: error.message || "Error registering user" });
    }
  };

export const login = async (req, res, next) => {
  try {
    const {email, password} = req.body;
    if(!email || !password){
      return res
        .status(400)
        .json({message: "Please fill the required fields!!!"});
    }
    const result= await AuthService.login({
      email,
      password

```

Figure 23 Auth Controller


```
// scheduleController.js
import ScheduleService from "../services/scheduleService.js";

export const createSchedule = async (req, res) => {
  try {
    const userId = req.user.userId;
    const {
      teamOneId,
      teamTwoId,
      scheduleStatus,
      date,
      scheduleType,
      location,
      createdFromClub,
      createdFromTournament,
    } = req.body;

    // Validation
    if (!teamOneId || !teamTwoId || !date || !scheduleType || !location) {
      return res.status(400).json({
        message: "teamOneId, teamTwoId, date, scheduleType, and location are required",
      });
    }

    // Must be created from either club or tournament
    if (!createdFromClub && !createdFromTournament) {
      return res.status(400).json({
        message: "Schedule must be created from either a club or tournament",
      });
    }

    const schedule = await ScheduleService.createSchedule(req.body, userId);
    res.status(201).json({
      message: "Schedule created successfully",
      schedule
    });
  } catch (error) {
    // ...
  }
}
```

Figure 24 Schedule Controller

```

import { lazy, Suspense } from "react";
import { BrowserRouter, Routes, Route, Navigate, useLocation } from "react-router-
import { SidebarProvider } from "../context/SidebarContext";
import { ThemeProvider } from "../context/ThemeContext";
import ProtectedRoute from "../components/Global/ProtectedRoute";
import ProtectedShell from "../components/Global/ProtectedShell";
import AppSkeleton from "../components/ui/AppSkeleton";
import PageTransition, { PageTransitionItem } from "../components/ui/PageTransition
import ParticlesBackground from "../pages/Player/Components/ParticlesBackground";

const Register = lazy(() => import("../pages/Authentication/register"));
const Login = lazy(() => import("../pages/Authentication/login"));
const ForgotPassword = lazy(() => import("../pages/Authentication/forgotPassword"));
const ResetPassword = lazy(() => import("../pages/Authentication/resetPassword"));
const MyClubs = lazy(() => import("../pages/Club/MyClubs"));
const ClubDetails = lazy(() => import("../pages/Club/ClubDetails"));
const Tournaments = lazy(() => import("../pages/Tournament/MyTournaments"));
const TournamentDetails = lazy(() => import("../pages/Tournament/TournamentDetails"));
const Dashboard = lazy(() => import("../pages/Player/Dashboard"));
const Discover = lazy(() => import("../pages/App/Discover"));
const Schedule = lazy(() => import("../pages/Schedule/Schedule"));
const ScheduleDetails = lazy(() => import("../pages/Schedule/ScheduleDetails"));
const Profile = lazy(() => import("../pages/Player/Profile"));
const Settings = lazy(() => import("../pages/App/Settings"));
const PaymentComponent = lazy(() => import("../components/payment/paymentForm"));
const Success = lazy(() => import("../components/payment/success"));
const Failure = lazy(() => import("../components/payment/failure"));

function PublicRouteElement({ children }) {
  return (
    <Suspense fallback={<AppSkeleton />}>
      <PageTransition>
        <PageTransitionItem>{children}</PageTransitionItem>
      </PageTransition>
    </Suspense>
  );
}

```

Figure 25 Frontend App Routing Skeleton

```

Frontend > src > services > JS api.js > ...
1  import axios from 'axios';
2
3  const api = axios.create({
4    baseURL : import.meta.env.VITE_API_BASE_URL || 'http://localhost:5555/api',
5    timeout:10000,
6    headers:{
7      'Content-Type':'application/json',
8    },
9  });
10
11 //Add token to every request (after login)
12 api.interceptors.request.use(
13   (config)=>{
14     const token = localStorage.getItem('token');
15     if(token){
16       config.headers.Authorization = `Bearer ${token}`;
17     }
18     return config;
19   },
20   (error) => Promise.reject(error)
21 );
22
23 //Global Error Handling
24 api.interceptors.response.use(
25   (response) => response,
26   (error) => {
27     if(error.response?.status === 401){
28       localStorage.removeItem('token');
29       window.location.href = '/login';
30     }
31     return Promise.reject(error);
32   }
33 )
34
35 export default api;

```

Figure 26 Frontend API Client with Auth Interceptors

10.2 User Survey Questions

In order to determine the particular needs of the FootStats system, the survey of the amateur players in the area, club administrators and tournament organizers was performed. The key questions included:

1. General Question:

- What is your existing position in sports community? (Player, Club Admin, Organizer)
- In which frequency do you play or organize matches/tournaments?

2. Current Challenges:

- What is your present practice with player line-ups and player statistics? (Manual notes, Excel, Verbal)
- What are the greatest challenges that you encounter in planning a tournament? (Time conflict, Administration of payments, Committee management)

3. Feature Expectations:

- Would you like to use digital system to pay tournament registration fees through Esewa/Khalti?
- What data are the most critical that you wish to monitor? (Goals, Assists, Cards, man of the match)
- The possibility to find local clubs and tournaments is also vital.

10.3 API Endpoints Specification

This section lists sample API routes used for backend communication between the React frontend and Node.js backend.

Method	Endpoints	Description
Auth		
Post	/api/auth/register	Register a new user profile
Post	/api/auth/login	Authenticate user and return JWT
Get	/api/clubs	Retrieve all clubs
Get	/api/tournaments	List all tournaments
Post	/api/payment/initiate	Initialize payment transaction
POST	/api/payment/verify	Verify payment callback from gateway
Get	/api/schedules	Get all schedules

Table 15 API Endpoints

10.4 User Stories

The SRS was used to derive the following user stories that would be used as the basis of development sprints:

- As a Player: I would desire to see my game history and statistics (goals/assists) so that I can check how I got better and better.
- As a Club Admin: I would wish to accept/ decline join requests in order to have a control over who plays on my team.
- As a Host of a Tournament: I do not want to manually create the fixtures as I want the system to run it automatically.
- As a Super Admin: I need to confirm tournaments prior to being live so as to ascertain that they are pure and correct.

11 Developed Project (Screenshots / System Demo / Wireframe)

This part gives the developed project of the FootStats in the form of screenshots and materials of system demonstration. All the screen shots focus on the significant elements on the system e.g., the login page, players dashboard, club management interface, tournament registration process and the payment gateway integration. These business deliverables allow readers to have a vision of how the system is operated based on experiences and give these outputs evidence of success that the solution offered was able to execute.

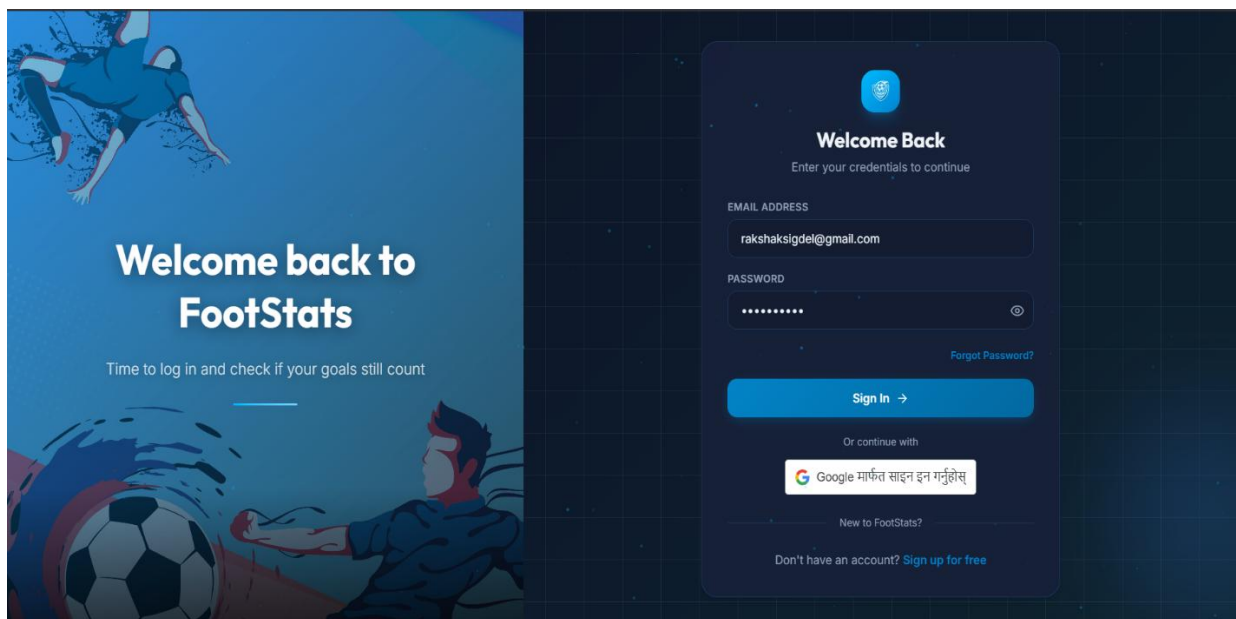


Figure 27 Login page

In the above screen shot, the user can see the primary authentication window that is log in page.

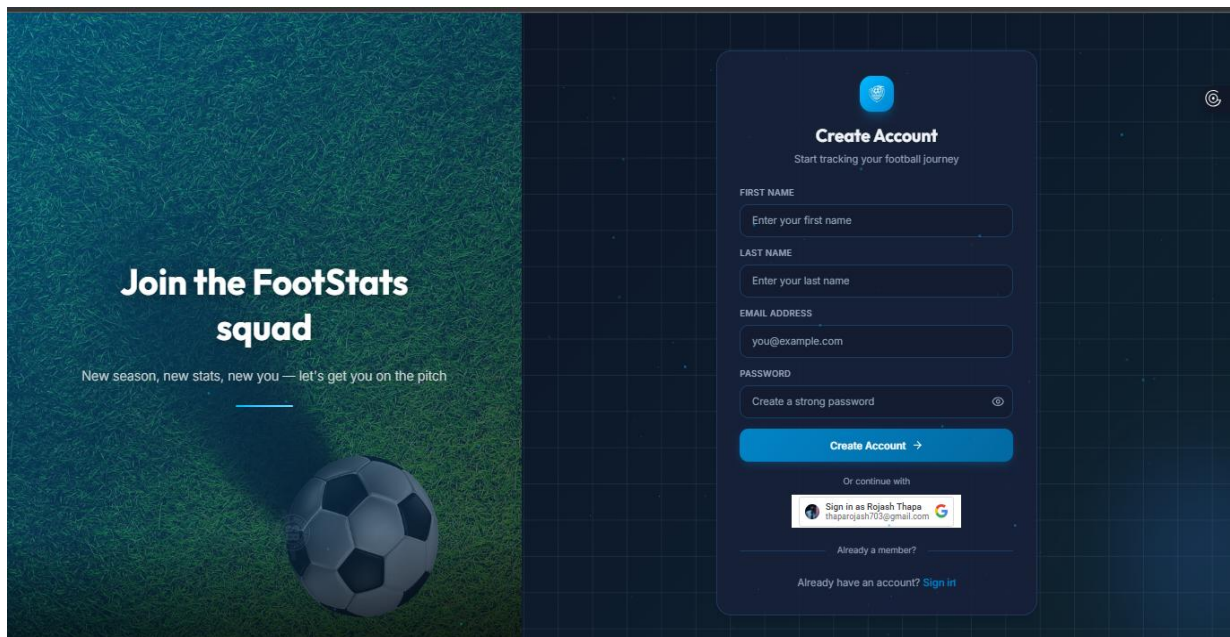


Figure 28 Register Page

In the above screen shot, the user can see the page where they can enter their details to register in the application.

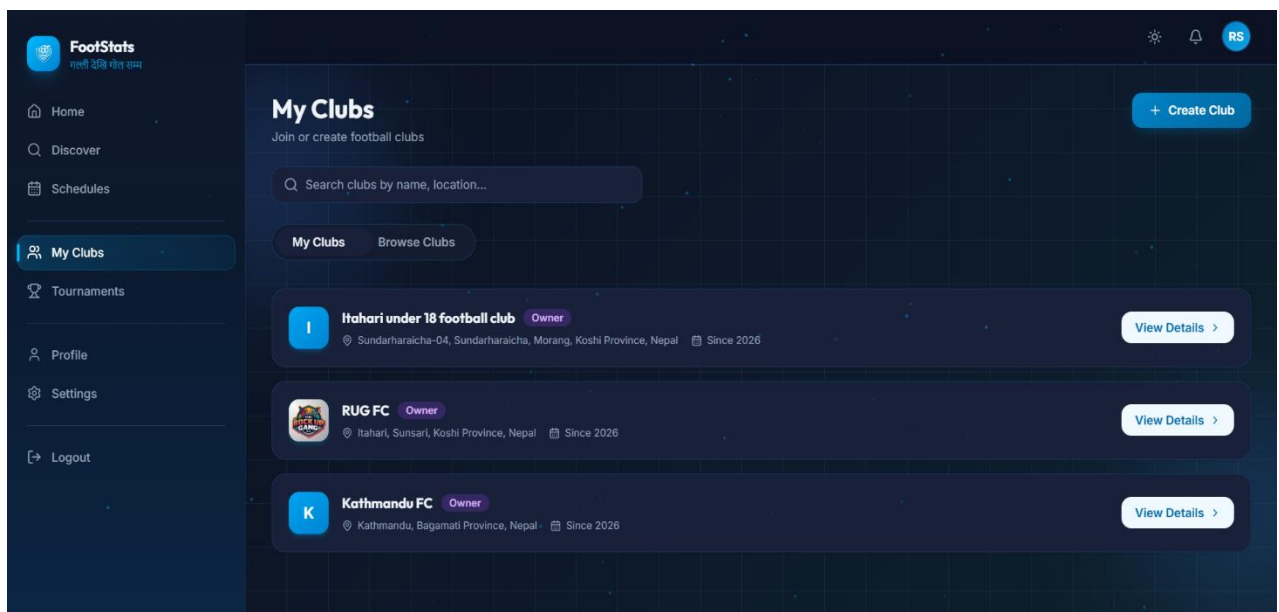


Figure 29 Create Club or Join

In the above screenshot, the user can create a club or can join other club.

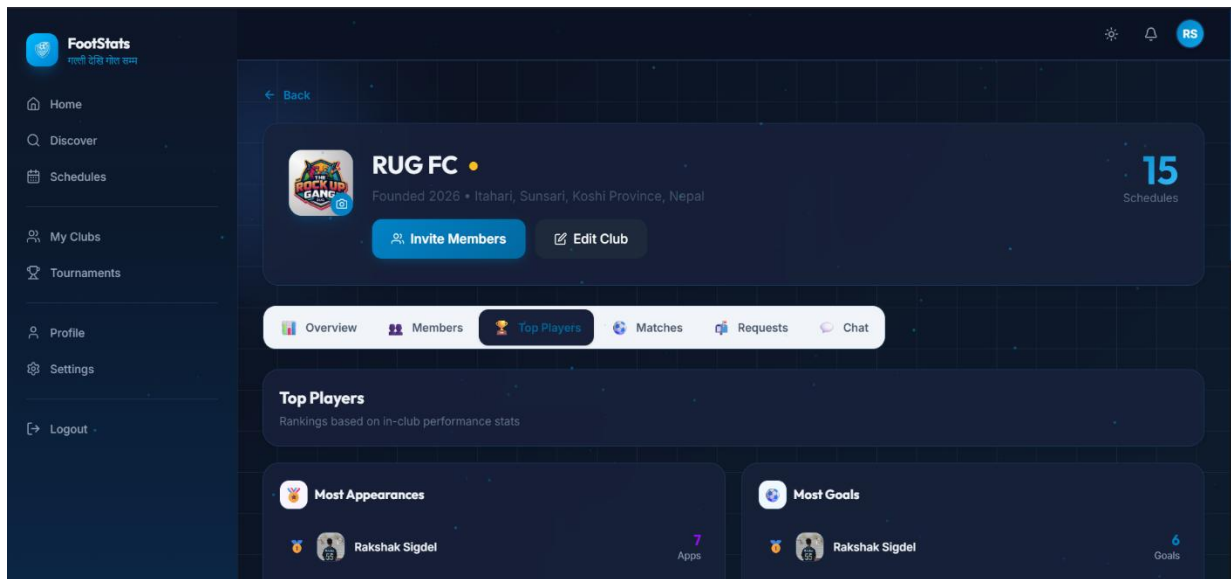


Figure 30 Club Details

In the above screenshot the user can view the club details.

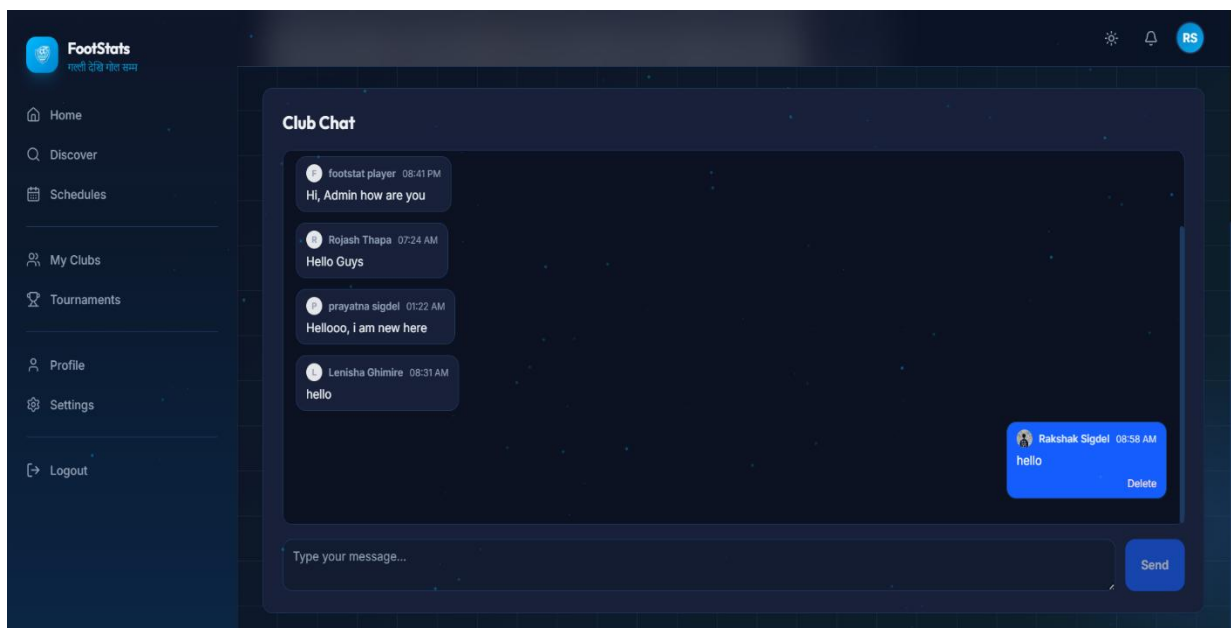


Figure 31 Chat Section

In the above page. the user is able to chat with the members of the club which they are joined.

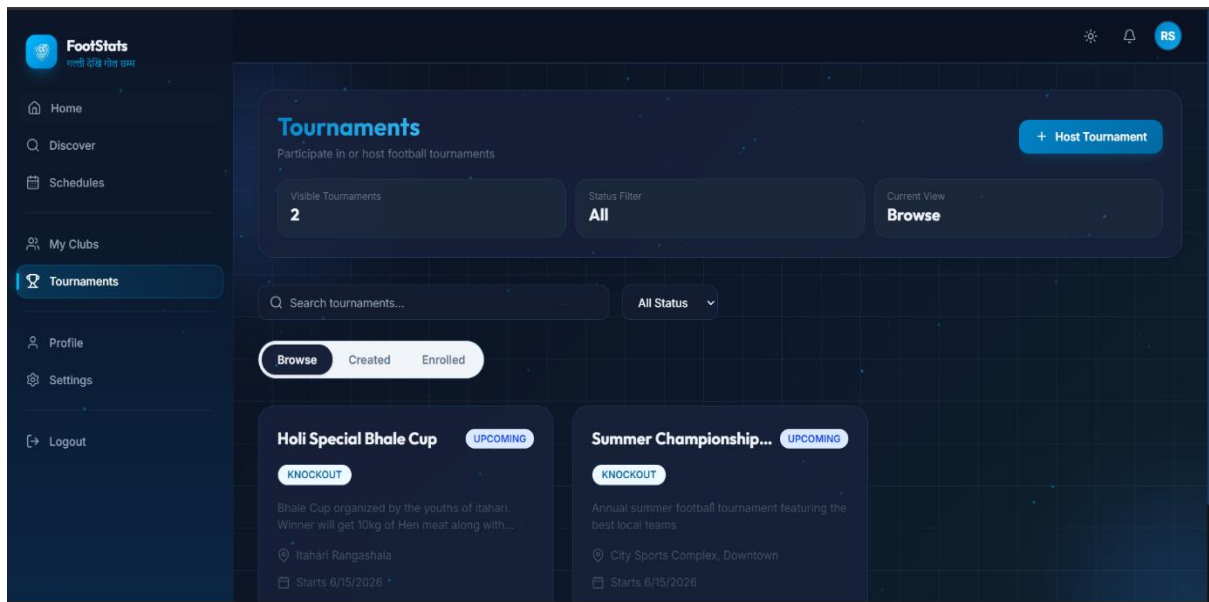


Figure 32 Host or Join Tournament

In the above page, the user can host or view tournament with their club.

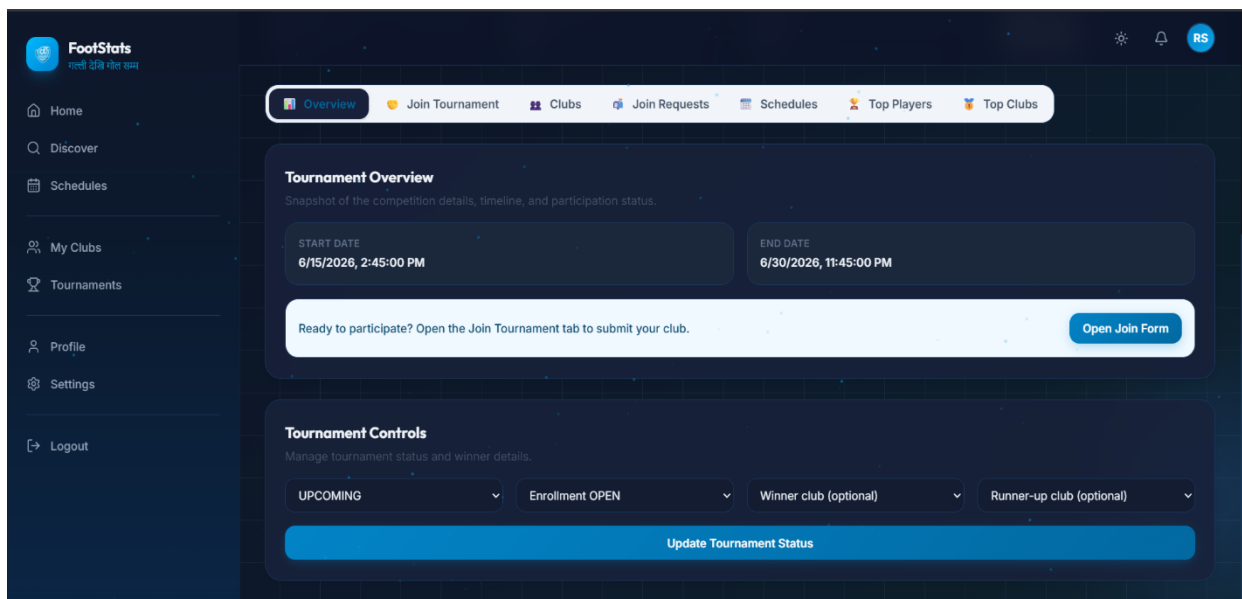


Figure 33 Tournament Management

In the above page, the admin or host of the tournament can manage their tournament.

Figure 34 Create Schedule

In the above page, the club admin or tournament host can create an schedule for match with other teams.

Figure 35 Manage line-ups

In the above page, the schedule creator can manage the line-ups for the team.

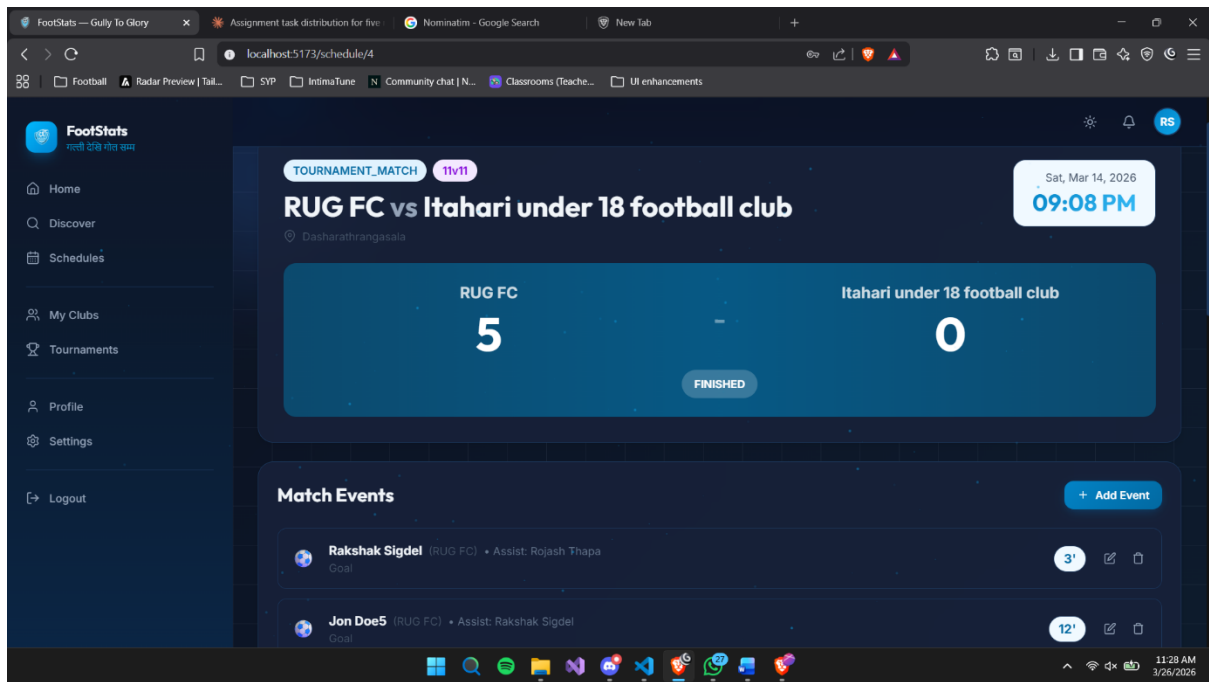


Figure 36 Update Match Event

In the above page, the schedule creator can update the match event after the game is finished.



Figure 37 Player Statistics Chart

The above page shows the Statistics of the player and the overall performance of the player.

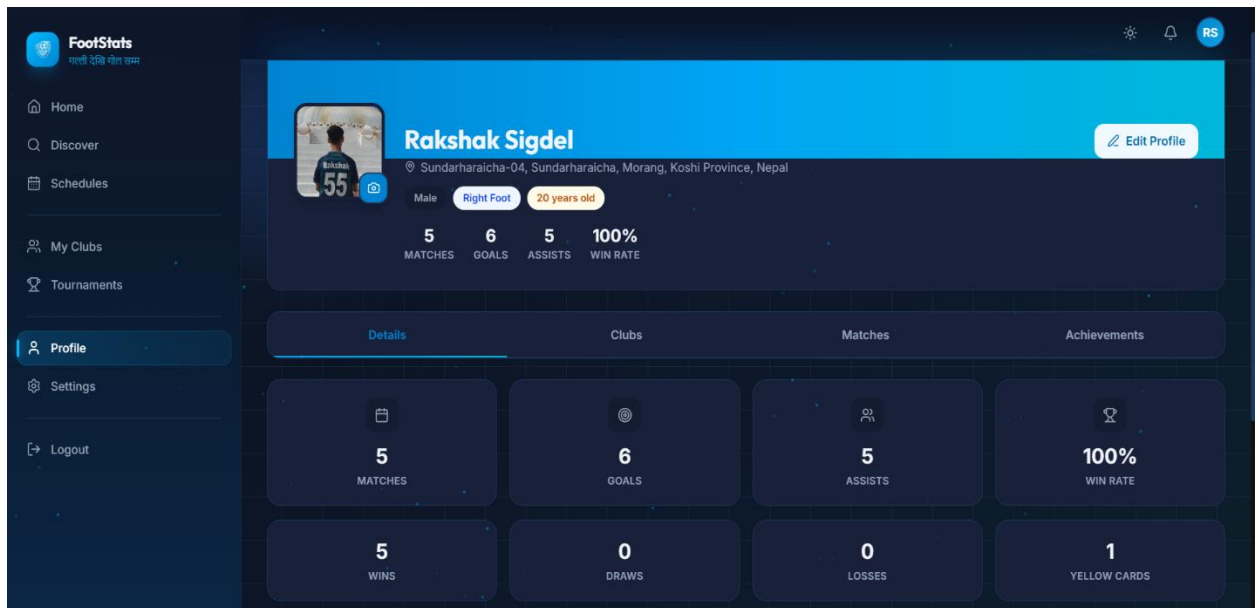


Figure 38 Profile View

In the above page, the user can view the profile of other players.

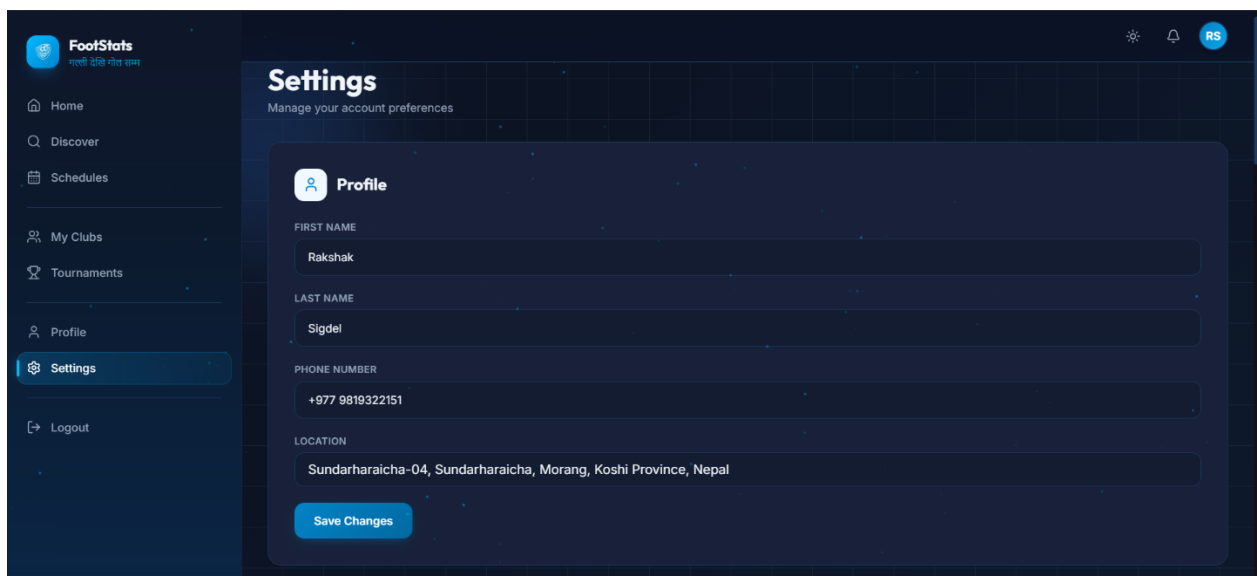


Figure 39 Update Profile

In the above page, the user can update their profile.

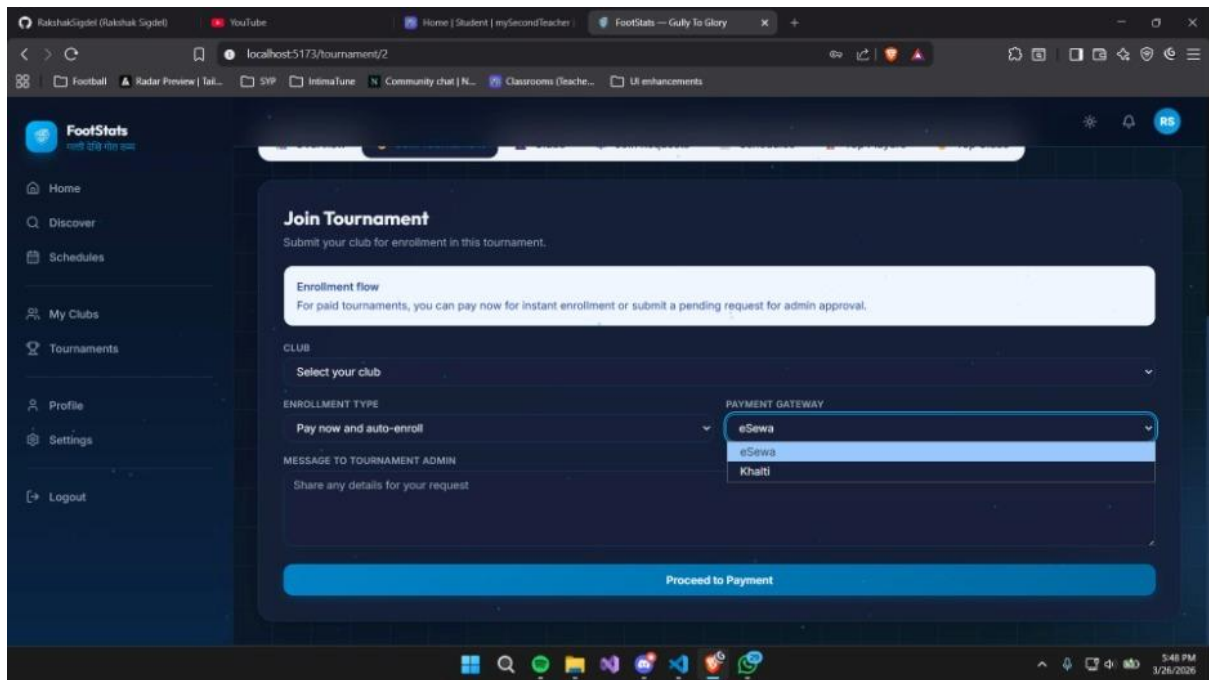


Figure 40 Payment

In the above page, the club admin can pay the tournament fee for registering into the tournament.